Flaris Virtual Machine — Bytecode Specification

The Flaris programming language · flaris-lang.org · github.com/flaris-lang/flaris · flaris-lang.org/discord

Version 1.0.2.0 **Generated** 2026-08-25 **License** see repository

This document is the normative reference for implementers of independent Flaris compilers, loaders, disassemblers, or virtual machines. It describes the on-disk bytecode format and the observable execution semantics of every instruction. The companion diagram flx-format.svg illustrates the container layout; where diagram and text disagree, this text is authoritative.

---

1. Introduction

1.1 Scope

The specification covers:

- the `.flx` binary container: header, signature, string pool, chunks, bundles
- the serialization of constants, functions and classes
- the instruction encoding and the semantics of all 174 opcodes
- the abstract machine: value system, operand stack, call frames, fibers, exception handling
- the validation rules a conforming loader must apply before execution

It does **not** cover: the Flaris source language (see guide.md and reference.md), the built-in function library (see reference.md), the FFI plugin ABI (see ffi.md), or the optional JIT IR side-channel (a conforming VM may ignore it; see §3.7).

1.2 Conformance terminology

- **MUST / MUST NOT** — required for conformance. A loader that skips a MUST validation rule accepts malformed bytecode and is non-conforming.
- **SHOULD** — recommended; deviation must not change observable program behavior.
- **implementation note** — describes the reference implementation (`flarisvm`); an independent implementation may differ as long as observable semantics match.

1.3 Versioning

Constant	Value	Meaning
`SYS_VERSION` / `CHUNK_VERSION`	`0x01000009` (1.0.0.9)	version written into produced chunks
`MIN_SUPPORTED_BYTECODE_VERSION`	`0x01000008`	oldest chunk version a loader MUST accept
`DEFAULT_LIB_VERSION`	`0x01000000`	default module version when unspecified

Version words pack four bytes `major.minor.patch.revision`, most-significant byte first within the 32-bit value (e.g. `0x01000008` = 1.0.0.8).

Opcode numbering is dense (Appendix A): inserting or removing an instruction renumbers every following opcode, and `MIN_SUPPORTED_BYTECODE_VERSION` is bumped in the same change. New opcodes **appended before `OP_LAST`** keep all prior numbers stable and only bump `SYS_VERSION` (older VMs reject the newer files via the version ceiling; 1.0.0.9 appended `OP_CONCAT_N` and `OP_NIL_LOCAL` this way). A loader MUST reject a chunk whose version is below the floor or above its own `SYS_VERSION`.

---

2. Notation and conventions

- **Byte order.** All multi-byte integers in the container *and* in instruction operands are **little-endian**, without exception.
- **Types.** `u8` / `u16` / `u32` / `u64` are unsigned little-endian integers of that width; `i8` / `i16` / `i32` / `i64` are two's-complement signed; `f32` / `f64` are IEEE 754 binary32/binary64.
- **Instruction syntax.** An instruction is written `MNEMONIC <operand:type> <operand:type> …`. The opcode itself is always one byte. Operands follow immediately, in the order listed.
- **Stack notation.** Stack effects are written `…, a, b → …, r`: the right end is the top of stack (TOS). `→ …` alone means the instruction pushes nothing.
- **`local[n]`** is slot *n* of the current call frame. **`TOS`** is the top of the operand stack. **`ip`** is the byte offset of the *next* instruction unless stated otherwise.
- Hash values are 64-bit **xxHash64 with seed 0** over the identifier's UTF-8 bytes unless stated otherwise (§4.6).

---

3. The abstract machine

3.1 Overview

The Flaris VM is a **stack machine** executing one **fiber** at a time on a single OS thread. A fiber owns:

- an operand stack of `Object*` values (initial `DEFAULT_STACK_SIZE` = 4096 slots, hard cap `MAX_STACK` = 65535),
- a call-frame stack (initial `DEFAULT_CALLFRAMES` = 64, hard cap `MAX_CALL_FRAMES` = 1024),
- an exception-context stack of `MAX_FL_EXCEPTION_LEVELS` = 24 nested try/catch/finally regions,
- a one-slot result, a bounded FIFO mailbox, and scheduling state.

Each call frame holds: the executing chunk, the instruction pointer `ip`, up to `MAX_LOCALS` = 128 local slots, the function object, its environment (for closures/globals), and the owning class for methods (`this` dispatch, `super`).

Scheduling is cooperative: the dispatch loop decrements a per-fiber quantum at scheduling checkpoints (loop back-edges and calls) and yields to the scheduler after `FIBER_QUANTUM` = 10000 checkpoints, or immediately at explicit `OP_YIELD` / `OP_AWAIT` / blocking-I/O suspension points. No OS threads are involved; bytecode never observes preemption in the middle of an instruction.

Implementation note (dispatch): the reference VM uses computed-goto dispatch over an `OP_LAST`-sized table (174 entries), caches `frame` / `constants` / `locals` / `code` / `ip` in locals, and re-derives them after any operation that can change the frame. The frame's stored `ip` is only guaranteed current at frame switches, raises and suspension points.

3.2 Calls, frames and returns

Call-site stack layout. Arguments are pushed left-to-right, then the callee on top; `OP_CALL <argc:u8>` consumes all of them and leaves the return value. Method forms (`OP_INVOKE` family) take the receiver from the stack (or a local/global for fused forms) and leave arguments on the stack for the callee.

Arity. A function declares `arity` parameters of which the trailing `optArgs` are optional (`VAL_OPTIONAL` bit). A call MUST satisfy `arity − optArgs ≤ argc ≤ arity`; violations raise `InvalidArgs`. Omitted optionals are filled with `nil` (the real nil value, never an empty slot). Typed parameters (declared type masks) are checked at call time unless the instruction is `OP_CALL_TYPED` (compiler-proven); `nil` passes any declared type. Arguments bind to `locals[0..argc−1]`; for methods, `locals[0]` is the receiver (`this`) and arguments shift up by one.

Frame entry. Exceeding the frame budget raises a catchable `StackError`; on entry the VM reserves `maxStack` (§8.3) operand slots so the body cannot overflow mid-execution. A frame owns its locals and function reference; on return every local and the function reference are released and any try-contexts belonging to the frame are discarded.

Tail calls. `OP_TAIL_CALL` / `OP_TAIL_SELF` replace the current frame instead of pushing (constant call depth); frame-budget overflow is therefore only reachable through non-tail recursion. Tail dispatch to a non-plain callee (builtin, bound method, class, …) degrades to call-then-return.

Fiber completion. When the last frame returns, the fiber finishes; its return value transfers to the awaiting fiber's result slot if one exists, otherwise it is preserved on the fiber for a later `await`.

Non-callable callees print a console error and produce `nil` — they do **not** raise.

3.3 Fibers, yield and await

Fiber states: `NEW`, `RUNNING`, `SUSPENDED`, `FINISHED`, `WAIT_IO`, `YIELD`, `WAIT_FIBER`. Scheduling checkpoints (quantum decrement) occur at backward jumps, taken conditional branches, iterator back-edges, and call/return boundaries; quantum expiry silently re-queues the fiber (no observable state change). Semantics of `OP_YIELD` / `OP_AWAIT` are in their Chapter 7 entries; the model is: `await fiber` parks the awaiter until the target delivers its result (a finished target delivers immediately); `yield v` delivers `v` to a resumer/awaiter if one is attached, else discards it. Exceptions never cross fiber boundaries.

3.4 Exceptions

Raises are soft. Raising does not unwind at the raise site: the VM builds an `Exception` instance, stores it as the fiber's current exception, sets the fiber's exception flag, and delivery happens at the next dispatch checkpoint. Consequences an implementer MUST reproduce: an instruction that raises abandons its remaining work, and a raise is only catchable if the fiber has an active try context — an uncaught raise prints the stack trace and terminates the **process** (exit code = the exception code).

Try contexts. `OP_TRY_BEGIN` pushes a context recording: catch landing pad, finally landing pad, current frame count, current stack depth, the local slot the caught exception binds to, and the owning frame's live local count. Nesting deeper than 24 (`MAX_FL_EXCEPTION_LEVELS`) raises `RuntimeError`. The context carries a state (`in-try` / `in-catch` / `in-finally`) and a pending action (`none` / `raise` / `return` / `break` / `continue`) so that `break` / `continue` / `return` /re-raise all route **through** the finally body.

Delivery. On delivery the VM walks to the innermost context: drains the operand stack to the recorded depth, discards frames above the recorded frame count, resets locals declared after `TRY_BEGIN` to `nil`, then enters the catch (pushing the exception object for `OP_CATCH_BEGIN` to bind) if the context was still in its try body and has a catch — otherwise it enters the finally with a pending re-raise. A raise **inside a finally** discards that context's pending action and propagates outward (the new exception wins). Contexts survive their catch body and pop only at `OP_FINALLY_END`.

Exception codes. The names used by the "Traps" clauses in Chapter 7 map to these numeric codes — observable as the `Code` field of a caught exception, and as the process exit code when uncaught. (Codes marked ° are raised by built-in functions and runtime subsystems rather than by the core instructions; 13 is unassigned.)

Code	Name (Chapter 7)		Code	Name (Chapter 7)
0	`NullPointer`		12	`StackError`
1	`DivisionByZero`		14	° unsafe-mode required
2	`ModuloByZero`		15	`NestingError`
3	`InvalidArgs`		16	° illegal instruction
4	`IndexOutOfBounds`		17	° executable-memory OOM
5	° I/O error		18	° out of fibers
6	`RuntimeError`		19	`ConstAssign`
7	° invalid state		20	° checksum mismatch
8	° out of memory		21	`ClassNonStaticCall`
9	° memory access		22	`TypeMismatch`
10	° array size		23	° assertion failed
11	`GuardNull`		24	`FieldUndeclared`

Const-check caveat. `EnsureMutable` sites (e.g. `OP_SET_GLOBAL`, array fast-path stores, this-field compound assigns) raise `ConstAssign` softly **and the mutation still proceeds**; the exception is delivered at the next checkpoint. `OP_SET_OBJ_L/LL` and `OP_SET_PROPERTY` hard-check instead (no mutation). Independent implementations MUST match this per-site behavior for observable compatibility.

3.5 Modules

A module executes its top-level chunk once; `OP_EXPORT` instructions populate its export table. Export lookup, missing-export errors and the module-object property protocol are specified in the Chapter 7 entries for `OP_EXPORT` and `OP_GET_PROPERTY`. Modules are identified by name + version + fingerprint (§5.3); `library(name, version, hash)` pins all three.

3.6 Self-patching instructions

Six `this.*` instruction forms rewrite themselves in place, at first execution, from hash-keyed lookups into resolved-index fast forms of the **same byte length**:

Emitted form (compiler)	Patched form (runtime)
`OP_GET_THIS_PROP`	`OP_GET_THIS_SLOT` / `OP_GET_THIS_CONST`
`OP_SET_THIS_PROP`	`OP_SET_THIS_SLOT`
`OP_THIS_ARITH_L` / `OP_THIS_ARITH_C`	`OP_THIS_ARITH_SLOT_L` / `OP_THIS_ARITH_SLOT_C`
`OP_THIS_INVOKE`	`OP_THIS_INVOKE_SLOT`
`OP_GET_THIS_ARR_L`	`OP_GET_THIS_ARR_SLOT_L`

(`OP_INVOKE_INSTANCE` additionally re-writes its inline u16 cache operand.) Patching is valid because class member layouts are base-first (§4.5): a resolved index is correct for the defining class and every subclass. Patched operands are resolved indices, **not** constant-pool indices. Patches apply only for non-native classes and happen in the shared in-memory chunk. A static compiler SHOULD emit only the hash-keyed forms; the slot forms are legal in a `.flx` but their indices are validated only at run time (guarded per access). An independent VM MAY treat self-patching as an optional optimization — executing the hash-keyed forms every time is observably equivalent.

3.7 JIT IR side-channel

A chunk may carry a serialized JIT IR section (§5.5) and functions carry JIT-related flags. These are optimization hints only: a conforming VM MAY ignore all of them. The reference VM guards JIT entry behind exact runtime type checks and falls back to interpretation, so JIT presence is never observable in program semantics (timing aside).

---

4. The value system

4.1 Value types

Every stack slot, local, constant and container element is a *value*. The dynamic type of a value is one of the `ObjectType` bits below. Type words are **bitmasks** (`ValMask`, 32-bit): a single concrete value has exactly one bit set, while declared types (function signatures, typed arrays, `OP_EXPORT` annotations) may union several bits.

Basic storable types (bits 0–8) — permitted as typed-array element types:

Bit	Name	Value	Meaning
0	`VAL_NIL`	`0x00000001`	the single value `nil`
1	`VAL_CHAR`	`0x00000002`	Unicode scalar, 32-bit
2	`VAL_FLOAT`	`0x00000004`	IEEE 754 binary64
3	`VAL_INT`	`0x00000008`	signed 64-bit two's-complement
4	`VAL_STRING`	`0x00000010`	mutable byte string (UTF-8 by convention, no embedded NUL)
5	`VAL_OBJECT`	`0x00000020`	hash-keyed map (string keys)
6	`VAL_ARRAY`	`0x00000040`	dynamic array
7	`VAL_BOOL`	`0x00000080`	`true` / `false`
8	`VAL_BLOCK`	`0x00000100`	raw memory block / typed buffer (element size 1–255 bytes)

Bits 9–15 are reserved for future basic types.

Complex runtime types (bits 16–28) — never element types:

Bit	Name	Value
16	`VAL_FUNCTION`	`0x00010000`
17	`VAL_BUILTIN_FUNCTION`	`0x00020000`
18	`VAL_MODULE`	`0x00040000`
19	`VAL_EXCEPTION`	`0x00080000`
20	`VAL_BOUND_METHOD`	`0x00100000`
21	`VAL_POINTER`	`0x00200000` (internal)
22	`VAL_FIBER`	`0x00400000`
23	`VAL_FFI_FUNCTION`	`0x00800000`

Bit	Name	Value
24	`VAL_CLASS`	`0x01000000`
25	`VAL_INSTANCE`	`0x02000000`
26	`VAL_STREAM`	`0x04000000`
27	`VAL_UNKNOWN`	`0x08000000`
28	`VAL_OPTIONAL`	`0x10000000` (parameter may be omitted)

`VAL_ANY` = `0xFFFFFFFF`. Bits 29–31 are reserved.

Typed arrays. A typed array stores its element type in the low 16 bits of its own type word alongside `VAL_ARRAY`, unshifted: `type = VAL_ARRAY | elementBits`. Example: `[int]` has type `0x00000048` (`VAL_ARRAY | VAL_INT`). An array of float literals is stored *flat* (`double[]` backing, no per-element boxes); writes coerce to float (implementation note: flag `OBJECT_FLAG_FLAT_FLOAT`).

4.2 Numeric model

- **int** is signed 64-bit two's-complement. Arithmetic wraps (implementation note: small ints -2^{60} … $2^{60}-1$ are pointer-tagged immediates, larger ones heap-boxed; the distinction is unobservable).
- **float** is IEEE 754 binary64. Conversions float→int: NaN → 0, ≥ 2^{63} → `INT64_MAX`, < -2^{63} → `INT64_MIN`, otherwise truncation toward zero.
- **Division `/` and power `^^` of int by int produce int** (truncating); the result is float only if at least one operand is float. `INT64_MIN / -1` and `INT64_MIN % -1` are defined (no trap; result `INT64_MIN` / `0`).
- **Shifts** mask their count to 0–63 (`count & 63`). Left shift is computed in unsigned arithmetic (two's-complement wrap, defined for negative operands); right shift is arithmetic (sign-extending).
- **char** is an unsigned 32-bit Unicode scalar; it participates in arithmetic as an integer and compares numerically.
- **Approximate equality** (`OP_APPROX_EQ`): relative epsilon `1e-2`, absolute epsilon `1e-9`.
- Float formatting uses `%.17g` (round-trip precision).

4.3 nil, truthiness and identity

`nil` is a real value (a singleton), not a null pointer. Reference types may hold `nil`. Truthiness rules are used by `OP_JUMP_IF_FALSE` / `OP_JUMP_IF_TRUE`, `OP_NOT` and the logical operators (see §7, OP_JUMP_IF_FALSE, for the normative table). `OP_CMP_IS` compares identity (same object), not structural equality.

4.4 Strings

Strings are **mutable in place**, length-tracked byte sequences with no embedded NUL bytes (`length` is always `strlen`-derived). Implementation notes: strings of ≤ 7 bytes use small-string optimization; a string caches its 64-bit hash and must rehash after in-place writes.

4.5 Objects, classes and instances

- A plain **object** (`VAL_OBJECT`) is a hash map with string keys.
- A **class** carries an ordered, kind-tagged member table (`FIELD`, `METHOD`, `CONST`) declared base-first; inheritance nests at most `MAX_PROTO_NEST` = 8 deep. A resolved layout is cached per class: an inherited member keeps its index and field slot in every subclass.
- An **instance** is slot-backed: a flat array of field values, one slot per `FIELD` member, addressed by the field-slot index resolved through the class layout. Instances are **sealed** — only declared fields exist.
- Property names are addressed by **64-bit hash** at runtime (§4.6); there is no interned-symbol table.

4.6 Name hashing

Identifiers (globals, property keys, method names) are referenced in bytecode operands as indices into the chunk's constant pool; the runtime resolves them by **xxHash64(name, seed = 0)** over the identifier bytes. The empty string hashes to `0xEF46DB3751D8E999`. There is no separate table of well-known names to implement: every hash is derivable from the identifier text (e.g. `Constructor` hashes to `0x9A9C0E93AC390B36` — a useful self-test vector). An independent implementation MUST reproduce xxHash64 exactly to interoperate with compiled chunks and native modules.

4.7 Reference counting and cycles

Memory is reclaimed by **pure reference counting** with deferred frees. There is no tracing collector and no cycle collector: a reference cycle is never reclaimed. This is an observable property of the machine (destructor timing, `--mem` reporting) and a documented language-level constraint, not a defect an implementation must reproduce bug-for-bug; an implementation MAY reclaim cycles but SHOULD document the deviation.

---

5. `.flx` container format

5.1 File layout

```
offset 0      FLS header, 192 bytes                    (§5.2)
offset 192    [bundles only] TOC: u16 count + entries  (§5.7)
              main section:  string pool  (§5.4)
                             top-level chunk (§5.5)
              [bundles only] dependency sections
```

A non-bundle file's main section MUST start exactly at offset 192. All region lengths are derived (`end − start`); none is stored explicitly.

5.2 The FLS header (192 bytes)

Magic is the four ASCII bytes `FLS2`. Total header size: 192 bytes (`FLS_HEADER_SIZE`).

Offset	Size	Field	Type	Meaning
0	4	`magic`	raw	`"FLS2"` = `46 4C 53 32`
4	4	`version`	u32	**library version**, packed `0xMMmmppbb` (major.minor.patch.build). Default `0x01000000` (1.0.0.0). Informational: not validated at load
8	4	`compilerVersion`	u32	**bytecode version** = `SYS_VERSION`. Loader MUST enforce `MIN_SUPPORTED_BYTECODE_VERSION ≤ compilerVersion ≤ SYS_VERSION`
12	32	`name`	UTF-8	NUL-terminated output basename *without extension*, ≤ 31 chars. Part of the signed/fingerprinted bytes: renaming the output changes the fingerprint
44	4	`flags`	u32	bit 0 = `FLAG_BUNDLE`; all other bits reserved (write 0)
48	8	`entry`	u64	xxHash64 of the entry function name — `0xC6783A229050BEDC` (`"Main"`) if present, `0` for a library with no entry point
56	4	`mainSectionStart`	u32	absolute offset; MUST be 192 for non-bundles, `192 + 2 + depCount·152` for bundles
60	4	`stringPoolStart`	u32	= `mainSectionStart` in the current writer

Offset	Size	Field	Type	Meaning
64	4	`stringPoolEnd`	u32	
68	4	`codeStart`	u32	= `stringPoolEnd` in the current writer
72	4	`codeEnd`	u32	= `mainSectionEnd` in the current writer
76	4	`mainSectionEnd`	u32	
80	2	—		reserved, zero
82	1	`sigAlg`	u8	0 = unsigned, 1 = retired legacy Ed25519 (MUST be rejected), 2 = Ed25519 (§5.3). Any other value MUST be rejected
83	1	—		reserved, zero
84	32	`pubkey`	raw	Ed25519 public key; all-zero when unsigned
116	64	`signature`	raw	Ed25519 signature; all-zero when unsigned
180	12	—		reserved, zero

The four section-offset pairs are validated for ordering (`mainSectionStart ≤ stringPoolStart ≤ stringPoolEnd ≤ codeStart ≤ codeEnd ≤ mainSectionEnd ≤ fileSize`) but the parser reads pool and chunk **contiguously** from `mainSectionStart`; `codeStart` / `codeEnd` exist for validation and future extensibility only.

5.3 Signature and trust

Algorithm. Ed25519 (as in RFC 8032; the reference implementation uses Monocypher). The 64-byte secret key is `seed ‖ pubkey`.

Signed content. `digest = SHA-256( file[0..116) ‖ 64 zero bytes ‖ file[180..EOF) )` — the entire file with the 64-byte signature field masked to zero. `sigAlg` and `pubkey` are inside the signed range, so neither can be altered without breaking the signature.

Domain separation. The message actually signed is the 50-byte concatenation `"flaris.flx.sig.v1" ‖ 0x00 ‖ digest` (context string including its terminating NUL, 18 bytes, followed by the 32-byte digest).

Signing flow. The file is written with the signature field zeroed, hashed in one pass (equal to the masked digest by construction), signed, and the 64 signature bytes patched in place at offset 116.

Fingerprint. The module fingerprint (printed at compile time, pinned by `library(name, version, hash)` and by the package manager) is the SHA-256 of the **complete final file including the signature** — identical to `sha256sum file.flx`.

Verification verdicts and load policy. A loader MUST classify a file as one of: `UNSIGNED` (sigAlg 0, key/sig fields zero), `VALID` (cryptographically valid, signer not trusted), `TRUSTED` (valid and signer key in the trust set), `BAD` (signature check failed), `STRIPPED` (sigAlg 0 but key/signature bytes nonzero — tamper evidence), `UNSUPPORTED` (sigAlg 1 or any unknown value). `BAD`, `STRIPPED` and `UNSUPPORTED` MUST always be rejected. `UNSIGNED` and `VALID` are rejected only under `--require-signed`. All rejects exit with code 4. The reference trust set is one built-in anchor (`flaris-lang.org`, `b6e2237413be79854985a1d4650ddefca8bc9c517964e6b9814b887bc6b779be`) plus `~/.flaris/trusted_keys` (path override `$FLARIS_TRUSTED_KEYS`; one `hex64 [label]` per line, `#` comments, ≤ 256 keys, unreadable file = fail closed).

5.4 String pool

Each code region (the main section, and each bundled dependency section) begins with a shared string pool that deduplicates strings referenced more than once in the region's value tree:

```
u16   count               (0 … 65535)
count × {
    u8    isStatic        (informational; reader ignores it)
    u64   hash            (xxHash64 of body; TRUSTED by the reader, not recomputed)
    u32   len             (MUST be < MAX_STRING_SIZE)
    u8[len] body          (no NUL terminator)
}
```

Wherever a serialized value would be a string that has a pool slot, it is replaced by the 6-byte reference `u32 0xFFFFFF01` (`FLX_POOL_REF`) + `u16 slot`. `0xFFFFFF01` does not collide with any `ObjectType` tag, so the reader's type-tag switch stays unambiguous. `slot ≥ count` MUST be rejected.

Writer policy (informative): a string is pooled only if it occurs more than once (keyed on exact bytes + length + static flag — never on the cached hash, which may be stale for mutable strings); slots are assigned in first-seen order; only leaf strings are pooled (functions can't be — their nested chunk would need the pool before it is complete); hash-only keys (§5.6) are never pooled.

5.5 Chunk serialization

A *chunk* is one function body (the top level is a chunk too). On the wire:

Chunk header — 13 bytes:

Offset	Size	Field	Validation
0	1	magic	`0xA5` (`CHUNK_MAGIC`)
1	4	version u32	MUST satisfy `MIN_SUPPORTED_BYTECODE_VERSION ≤ v ≤ CHUNK_VERSION`
5	2	consts u16	MUST be ≤ `MAX_CONST_COUNT` (16384)
7	4	codelen u32	MUST be ≥ 1 and ≤ `MAX_CODE_SIZE` (10 MiB)
11	2	flags u16	bit 0 `CHUNK_FLAG_DBG`, bit 1 `CHUNK_FLAG_HAS_JIT_IR`

Chunk body, immediately following:

1. `consts` serialized constant values, in pool-index order (§5.6);
2. `codelen` bytes of code (§6);
3. if `CHUNK_FLAG_DBG`: `u8 localCount` (≤ 128), then `localCount × { u32 len; u8[len] name }` — local-variable names;
4. if `CHUNK_FLAG_HAS_JIT_IR`: `u8 regCount`, `u32 irLen` (≤ 10 MiB), `u8[irLen]` JIT IR. This section is an optional side-channel: a conforming VM MAY skip it entirely. The stored `regCount` MUST NOT be trusted (the reference VM recomputes it from the IR); semantically invalid IR is dropped (the function runs interpreted) and is *not* a load error.

`--strip` removes the debug section (clears `CHUNK_FLAG_DBG`) and suppresses the `OP_DBG_*` opcodes in code. There is no separate line table: source lines are the inline `OP_DBG_LINE` instructions.

5.6 Constant value serialization

General wire form: `u32 typeTag` + `u8 isStatic` + payload. The type tag is the value's `ObjectType` bit (§4.1). Exception: a pool reference (§5.4) is `u32 0xFFFFFF01` + `u16 slot` with **no** isStatic byte. The reader ignores `isStatic` in all cases.

Tag	Payload
`VAL_NIL 0x01`	none
`VAL_CHAR 0x02`	u32 codepoint
`VAL_FLOAT 0x04`	f64 (IEEE 754 bits, LE)
`VAL_INT 0x08`	i64
`VAL_STRING 0x10`	`u64 hash` + `u32 len` + `u8[len]` body. `len ≥ MAX_STRING_SIZE` MUST be rejected. If `len > 0` the reader recomputes the hash (stored hash ignored); if `len == 0` the stored hash is preserved — this is the **hash-only key** form
`VAL_OBJECT 0x20`	`u32 count` (< `MAX_ITEMS`), then `count ×` { key value — MUST decode to a string; value }. The writer emits pairs sorted by key hash for byte-deterministic output
`VAL_ARRAY 0x40`	`u32 length` (< `MAX_ITEMS`) + `u8 flat`. `flat = 1`: `length × f64` (flat float array, §4.1); `flat = 0`: `length` nested values
`VAL_BOOL 0x80`	u8 (0/1)
`VAL_BLOCK 0x100`	none — blocks do not serialize; reads back as `nil`
`VAL_FUNCTION 0x10000`	`u8 arity` + `u8 localCount` + `u32 flags` + `u32 returnType` + `16 × u32 argsTypes` + name (value-key, may be the nil sentinel) + **nested chunk** (§5.5, recursive). Reader MUST clamp `localCount ≤ 128`, `arity ≤ 16`, clear flags `FUNC_FLAG_IS_CLONE` + `FUNC_FLAG_ENV_OWNED`, and re-derive `FUNC_FLAG_TYPED_PARAMS` itself
`VAL_BUILTIN_FUNCTION` / `VAL_MODULE` / `VAL_EXCEPTION` / `VAL_BOUND_METHOD` / `VAL_POINTER`	none — read back as `nil`
`VAL_CLASS 0x1000000`	`u32 ownCount` (≤ `MAX_ITEMS`) + `u64 protoHash` (base-class name hash, 0 = none) + name (must be string) + `ownCount ×` { `u8 kind` (0 field / 1 method / 2 const; > 2 MUST be rejected), member name (must decode to string), member value }. Function members are back-linked to the class as owner
any other tag	MUST be rejected

Value-keys (object keys, class member names, function names) are ordinary serialized values, with two special forms: a NULL name is the 5-byte sentinel `[u32 VAL_NIL][u8 0]`, and under size optimization (`--small`) a string key may be written **hash-only**: `[u32 VAL_STRING][u8 static][u64 hash][u32 len=0]` — identity survives (runtime identity is the hash, §4.6), the text does not.

Nesting depth. The loader tracks one depth counter shared by chunk and value deserialization, rejecting at 64 (`MAX_CHUNK_DEPTH`). A nested function costs **two** levels (value → its chunk), so function nesting deeper than ~31 levels is not loadable.

5.7 Bundles

A bundle packs a program with its library dependencies into one file. Header bit `FLAG_BUNDLE` is set and a table of contents follows the header:

```
@192   u16 entryCount            (MUST be ≤ 1024)
@194   entryCount × BundleEntry  (152 bytes each)
       main section              (at hdr.mainSectionStart = 194 + entryCount·152)
       dependency sections
```

BundleEntry — 152 bytes:

Offset	Size	Field	Meaning
0	64	`name`	module name, NUL-terminated, `.flx` stripped

Offset	Size	Field	Meaning
64	4	`version` u32	dependency's library version (`0xMMmmppbb`)
68	8	`offset` u64	absolute file offset of the dependency's section
76	4	`size` u32	section length (pool + chunk)
80	32	`sha256`	whole-file SHA-256 of the original dependency `.flx` — the pin value for `library(name, ver, hash)`
112	1	`sigAlg` u8	dependency's original signature algorithm
113	32	`pubkey`	dependency's signing key (zero if unsigned)
145	7	—	reserved, zero

A dependency section is a byte-copy of `[mainSectionStart, mainSectionEnd)` of the original dependency file — its pool and chunk travel together, so pool references stay valid. Per-dependency bounds errors (offset/size outside the file, size 0) skip that dependency with an error rather than aborting the load. Under `--require-signed` the TOC-recorded `sigAlg` / `pubkey` hold bundled dependencies to the same trust policy as standalone files (the dependency's own header does not survive bundling); the TOC itself is covered by the bundle's outer signature. A bundle builder MUST refuse dependencies whose signatures verify as `BAD`, `STRIPPED` or `UNSUPPORTED`.

Load order. A bundled dependency may import another dependency in the same bundle. A loader MUST therefore make every TOC entry resolvable before executing any of them — registering each as an unloaded module and running it on first import — so that a dependency can import one listed after it. Trust (`--require-signed`) is decided for every entry at load time, whether or not anything imports it; content pins are checked before the dependency executes, using the `sha256` the TOC already carries. A dependency nothing imports is never executed. A cycle between bundled dependencies is detected and reported rather than recursing without bound; the import that closes the cycle fails, so the dependency holding it is left partially initialised.

Independently, a bundle builder SHOULD write TOC entries in dependency order — every dependency ahead of the entries that import it — so the bundle also loads on a runtime that walks the TOC front-to-back. Entries carry explicit offsets, so TOC order is independent of where the sections sit in the file. Order is advisory: it cannot be relied on, since nothing prevents a hand-built TOC from listing entries arbitrarily.

---

6. Instruction encoding

6.1 General form

An instruction is a 1-byte opcode followed by zero or more fixed-width operands in the order given by its definition (§7). Multi-byte operands are little-endian. There is no alignment: instructions pack back-to-back. Exactly two instructions have variable length (`OP_FN_CAPTURE` and `OP_JUMP_TABLE` , §6.5); every other instruction's length is a static function of its opcode.

6.2 Operand kinds

Kind	Width	Meaning
`const:u16`	2	index into the chunk constant pool (< `constCount` ≤ `MAX_CONST_COUNT` = 16384)
`key:u16`	2	constant-pool index of an identifier string (resolved to its xxHash64 at run time)
`slot:u8`	1	local-variable slot (< frame `localCount` ≤ `MAX_LOCALS` = 128)
`argc:u8`	1	argument count (≤ `MAX_CALL_ARGUMENTS` = 16)
`aop:u8`	1	arithmetic sub-operator (§6.3)

Kind	Width	Meaning
`cop:u8`	1	comparison sub-operator (§6.4)
`imm:i8` / `imm:i16` / `imm:i32`	1/2/4	signed integer immediate
`imm:f32`	4	IEEE binary32 immediate, widened to binary64 on push
`imm:u32`	4	unsigned immediate (`OP_IMM_CHAR` codepoint)
`off:u16`	2	branch displacement (forward for `JUMP*`, backward for `LOOP`), relative to the address after the operand (§6.6)
`addr:u16`	2	absolute code offset within the chunk
`type:u16` / `type:u32`	2/4	`ValMask` bits (§4.1): array element type, export type

6.3 Arithmetic sub-operators (`ArithOp`)

Used by `OP_ARITH_L/LL/LC/IMM8/ELL/ELC` , `OP_THIS_ARITH_*` :

Value	Name	Operator
0	`ARITH_ADD`	`+`
1	`ARITH_SUB`	`-`
2	`ARITH_MUL`	`*`
3	`ARITH_DIV`	`/`
4	`ARITH_MOD`	`%`
5	`ARITH_POW`	`^^`
6	`ARITH_AND`	`&`
7	`ARITH_OR`	`|`
8	`ARITH_XOR`	`^`
9	`ARITH_SHL`	`<<`
10	`ARITH_SHR`	`>>`

A loader MUST reject `aop` ≥ 11 (`ARITH_LAST`).

6.4 Comparison sub-operators

Used by `OP_CMP_JUMP_LL/LC/LC32` . **The branch fires when the comparison is false** (these ops fuse `compare + JUMP_IF_FALSE`):

Value	Name	Condition
0	`CMP_EQ`	`==`
1	`CMP_NEQ`	`!=`
2	`CMP_LT`	`<`
3	`CMP_GT`	`>`
4	`CMP_LE`	`<=`
5	`CMP_GE`	`>=`

6.5 Instruction lengths

Total length in bytes, opcode included. This table is normative for stepping over code; a decoder MUST treat a truncated instruction (operands running past `codelen`) as malformed.

Len	Instructions
1	all operand-less opcodes: `NOP` , `NIL` , `TRUE` , `FALSE` , `ONE` , `NEG_ONE` , `POP` , `DUP` , unary/binary arithmetic and comparisons (`NEGATE` … `CMP_IS`), `NULL_COALESCING` , `GET_LOCAL0-3` , `SET_LOCAL0-3` , `GET_INDEX` , `SET_INDEX` , `MAKE_CONST` , `RETURN` , `RETURN_NONE` , `RETURN_NIL` , `YIELD` , `AWAIT` , `BIND_THIS` , `TRY_END` ,

Len	Instructions
	`CATCH_BEGIN`, `CATCH_END`, `FINALLY_BEGIN`, `FINALLY_END`, `THROW`, `LEN`, `TYPE`, `IS_ARRAY`, `IS_OBJECT`, `HAS_KEY`, `TO_*` (all ten), `SUPER`, `GUARD`, `CONCAT`
2	`IMM8`, `GET_LOCAL`, `SET_LOCAL`, `INC_LOCAL`, `DEC_LOCAL`, `NIL_LOCAL`, `GET_ARR_LI`, `GET_INDEX_LOCAL`, `SET_INDEX_LOCAL`, `RETURN_L`, `CALL`, `CALL_TYPED`, `CALL_SELF`, `TAIL_SELF`, `TAIL_CALL`, `NEW`, `CONCAT_N`
3	`IMM16`, `CONSTANT`, `DEFINE_GLOBAL`, `GET_GLOBAL`, `SET_GLOBAL`, `FN`, `GET_PROPERTY`, `SET_PROPERTY`, `GET_THIS_PROP`, `SET_THIS_PROP`, `GET_THIS_SLOT`, `SET_THIS_SLOT`, `GET_THIS_CONST`, `JUMP`, `JUMP_IF_FALSE`, `JUMP_IF_TRUE`, `LOOP`, `ARITH_IMM8`, `ARITH_L`, `GET_ARR_LC`, `GET_ARR_LL`, `SET_ARR_LC`, `SET_ARR_LL`, `GET_BLK_LC`, `GET_BLK_LL`, `SET_BLK_LC`, `SET_BLK_LL`, `BUILD_OBJECT`, `PUSH_BUILTIN`, `CALL0_BUILTIN` … `CALL5_BUILTIN`, `DBG_LINE`, `DBG_FUNC_NAME`, `DBG_FILE_NAME`, `DBG_BREAK`
4	`INVOKE`, `THIS_INVOKE`, `THIS_INVOKE_SLOT`, `SUPER_INVOKE`, `CALL_GLOBAL`, `GET_THIS_ARR_L`, `GET_THIS_ARR_SLOT_L`, `GET_LOCAL_PROP`, `ARITH_LC`, `ARITH_LL`, `SET_ARR_LLC`, `SET_ARR_LLL`, `LOCAL_ARR_LC`, `LOCAL_ARR_LL`, `SET_OBJ_L`, `SET_BLK_LLC`, `SET_BLK_LLL`, `LOCAL_BLK_LC`, `LOCAL_BLK_LL`
5	`IMM32`, `IMM_F32`, `IMM_CHAR` (u32 codepoint), `ARITH_ELC`, `ARITH_ELL`, `SET_OBJ_LL`, `THIS_ARITH_L`, `THIS_ARITH_C`, `THIS_ARITH_SLOT_L`, `THIS_ARITH_SLOT_C`, `BUILD_ARRAY` (count:u16 + elemType:u16), `ITER_BEGIN`, `ITER_NEXT`, `TRY_LEAVE`, `ARITH_FMA_LLL`
6	`INVOKE_INSTANCE` (key:u16 argc:u8 slotCache:u16 — the cache field is runtime-managed scratch), `INVOKE_GLOBAL`, `CMP_JUMP_LL`, `CMP_JUMP_LC`, `TRY_BEGIN`
7	`FOREACH` (slot1:u8 slot2:u8 iterable:u8 index:u8 endAddr:u16; slot operands may be the sentinel `0xFF` = unused)
9	`CMP_JUMP_LC32`, `EXPORT` (aliasIdx:u16 nameIdx:u16 type:u32)
var	`FN_CAPTURE` = 4 + 3·count: `fnIdx:u16 count:u8`, then count × (`nameConstIdx:u16 parentSlot:u8`). `JUMP_TABLE` = 12 + 2·size: `min:i32 max:i32 size:u8`, then size × `caseAddr:u16`, then `defaultAddr:u16` (all addresses absolute)

6.6 Branch-target computation

Let `base` be the code offset immediately **after** the instruction's last operand byte.

- `OP_JUMP`, `OP_JUMP_IF_FALSE`, `OP_JUMP_IF_TRUE`, `OP_CMP_JUMP_LL/LC/LC32`: `target = base + off` (forward).
- `OP_LOOP`: `target = base − off` (backward).
- `OP_ITER_BEGIN`, `OP_ITER_NEXT`, `OP_FOREACH`, `OP_TRY_BEGIN` (`catchIp` / `finallyIp`), `OP_TRY_LEAVE`, `OP_JUMP_TABLE`: operands are **absolute** code offsets within the chunk.

---

7. Instruction set reference

Entries are in enum order (opcode numbers in Appendix A). Format:

> **`MNEMONIC`** `<operand:type> …` — total length Stack `…, a, b → …, r` (right end = TOS) · Operation · Traps

"Traps: —" means the instruction cannot raise. All raises are soft (§3.4). Fused instructions are semantically equivalent to the generic sequence they replace unless a difference is stated.

7.1 Stack and constants

`OP_NOP` — 1 byte. Stack `→`. No operation. Traps: —

`OP_CONSTANT` `<idx:const:u16>` — 3 bytes. Stack `→ v`. Pushes constant `idx`. A **string** constant is pushed as a fresh copy (strings are mutable; the pool object must not be aliased); other values are pushed by reference. Traps: —

`OP_NIL` / `OP_TRUE` / `OP_FALSE` — 1 byte. Stack `→ v`. Push `nil` / `true` / `false`. Traps: —

`OP_ONE` / `OP_NEG_ONE` — 1 byte. Push int `1` / `-1`. Traps: —

`OP_IMM8` `<v:i8>` / `OP_IMM16` `<v:i16>` / `OP_IMM32` `<v:i32>` — 2/3/5 bytes. Push the sign-extended int. Traps: —

`OP_IMM_F32` `<bits:u32>` — 5 bytes. Push float: the operand is IEEE binary32 bits, widened to binary64. Traps: —

`OP_IMM_CHAR` `<cp:u32>` — 5 bytes. Push char with codepoint `cp`. Traps: —

`OP_POP` — 1 byte. Stack `v →`. Discards TOS. Traps: `StackError` (underflow — unreachable in validated code).

`OP_DUP` — 1 byte. Stack `v → v, v`. Traps: `StackError`.

7.2 Arithmetic and logic

`OP_NEGATE` — 1 byte. Stack `v → -v`. Int negation wraps (`-INT64_MIN = INT64_MIN`); float negates. Traps: `InvalidArgs` on non-numeric.

`OP_NOT` — 1 byte. Stack `v → b`. Pushes `!truthy(v)` (§7.4 truthiness). Traps: —

`OP_BITWISE_NOT` — 1 byte. Stack `v → ~v`. Int only. Traps: `InvalidArgs`.

`OP_ADD` — 1 byte. Stack `a, b → r`. Lane selection, in order:

1. int ⊕ int (bool/char count as int): 64-bit wrapping add;
2. either float: binary64 add;
3. char + char: a **string** of the two codepoints (UTF-8);
4. object + object: clone of `a` with `b`'s properties merged over it;
5. array + array: clone of `a` with `b`'s elements appended; array + scalar: clone of `a` with the scalar appended;
6. anything else: **string concatenation** — both operands coerced to text. Traps: `NestingError` (deep container merge).

`OP_SUBTRACT` / `OP_MULTIPLY` — 1 byte. Stack `a, b → r`. Int lane wraps; float lane if either operand is float. Traps: `InvalidArgs` on non-numeric operands.

`OP_DIVIDE` — 1 byte. Stack `a, b → r`. int/int: truncating division, result int; `INT64_MIN / -1 = INT64_MIN`. Float lane if either is float. Zero divisor raises `DivisionByZero`. Note: on the generic path the zero check is `AsFloat(b) == 0.0`, and non-numeric values coerce to 0.0 — so dividing by a non-numeric value raises `DivisionByZero`, not a type error.

`OP_MODULO` — 1 byte. Stack `a, b → r`. int lane: C remainder (sign of the dividend: `-7 % 3 = -1`); `INT64_MIN % -1 = 0`. Float lane: `fmod`. Zero divisor raises `ModuloByZero` (same coercion note as `OP_DIVIDE`).

`OP_POWER` — 1 byte. Stack `a, b → r`. int ^^ int: computed via double `pow`, truncated back to int (§4.2) — negative exponents truncate to 0. Float lane if either is float. Traps: `InvalidArgs`.

`OP_BITWISE_AND` / **`OP_BITWISE_OR`** / **`OP_XOR`** — 1 byte. Stack `a, b → r`. Int-only bitwise ops. Traps: `InvalidArgs` ("requires integer operands").

`OP_SHL` / **`OP_SHR`** — 1 byte. Stack `a, b → r`. Shift count masked `& 63`; `<<` computed in unsigned (defined two's-complement wrap); `>>` is arithmetic (sign-extending). Int-only. Traps: `InvalidArgs`.

7.3 Comparison

`OP_IS_NIL` / **`OP_IS_NOT_NIL`** — 1 byte. Stack `v → b`. Tests `v` against the nil singleton. Traps: —

`OP_EQUAL` / **`OP_NOT_EQUAL`** / **`OP_LESS`** / **`OP_LESS_EQUAL`** / **`OP_GREATER`** / **`OP_GREATER_EQUAL`** — 1 byte. Stack `a, b → r:bool`. Rules:

- identity short-circuit: `a == a` is equal;
- numeric lane: int/float/char/bool inter-comparable (double compare if either is float, else int64);
- strings: equality is length + byte compare (the cached hash is only a fast-reject hint, never trusted alone); ordering is byte-lexicographic, shorter-is-less on a common prefix;
- arrays: element-wise structural equality; ordering by first differing element, else shorter-is-less;
- blocks: raw byte compare (equality);
- `nil == nil` is true;
- **incompatible types**: `!=` is true; every ordering comparison is false. Traps: `NestingError` beyond depth 1024 (`MAX_COMPARE_DEPTH`).

`OP_APPROX_EQ` (`~=`) — 1 byte. Stack `a, b → r:bool`. string ~= string: case-insensitive equality. Numeric: NaN never equal, infinities equal only same-signed, else `|a−b| ≤ 1e−9 + 1e−2·max(|a|,|b|)`. Other operands → false. Traps: —

`OP_CMP_IS` — 1 byte. Stack `a, b → r:bool`. If either operand is a class: instance-of / subclass-of test on the other (proto chain, ≤ 8 deep), order-independent. Otherwise plain identity `a == b`. Traps: —

7.4 Control flow

`OP_JUMP_IF_FALSE` / **`OP_JUMP_IF_TRUE`** `<off:u16>` — 3 bytes. Stack `c →`. Branch forward (§6.6) when `c` is falsy / truthy. **Truthiness** (normative): `nil` → false; bool → itself; int/float/char → `≠ 0`; string → non-empty; array → `length > 0`; object → has ≥ 1 property; every other type (functions, classes, instances, fibers, …) → true. Traps: —

`OP_JUMP` `<off:u16>` — 3 bytes. Unconditional forward branch. Traps: —

`OP_LOOP` `<off:u16>` — 3 bytes. Unconditional **backward** branch; the mandatory loop back-edge checkpoint (quantum, §3.3). Traps: —

`OP_NULL_COALESCING` — 1 byte. Stack `a, b → r`. `r = a` if `a` is non-nil (0, `""`, `false` are kept), else `r = b`. Not truthiness-based. Traps: —

`OP_JUMP_TABLE` `<min:i32> <max:i32> <size:u8> size×<case:addr:u16> <default:addr:u16>` — 12 + 2·size bytes. Stack `v →`. Coerces `v` to int; if `min ≤ v ≤ max` and `v − min < size` jumps to `case[v − min]`, else to `default`. All targets absolute. Traps: —

`OP_CMP_JUMP_LL` `<slotA:u8> <slotB:u8> <cop:u8> <off:u16>` — 6 bytes. Stack `→`. Compares `local[slotA]` with `local[slotB]` per `cop` (§6.4) and branches forward **when the condition is false**. Zero stack traffic. Equivalent to `GET_LOCAL a; GET_LOCAL b; CMP; JUMP_IF_FALSE`. Traps: `NestingError` (deep structural compare).

`OP_CMP_JUMP_LC` `<slot:u8> <cop:u8> <imm:i8> <off:u16>` — 6 bytes. As `OP_CMP_JUMP_LL` with an i8 immediate right operand. If the local is not a small int, the comparison is performed as `AsFloat(local)` vs `(double)imm` — non-numeric locals coerce to 0.0 rather than raising. Traps: —

`OP_CMP_JUMP_LC32` `<slot:u8> <cop:u8> <imm:i32> <off:u16>` — 9 bytes. `OP_CMP_JUMP_LC` with an i32 immediate. Traps: —

7.5 Globals and locals

`OP_DEFINE_GLOBAL` `<key:u16>` — 3 bytes. Stack `v →`. Defines the name in the current environment. Scalar values are stored as private copies (value semantics); containers by reference. Redefinition/shadowing prints a warning but succeeds. Traps: —

`OP_GET_GLOBAL` `<key:u16>` — 3 bytes. Stack `→ v`. Resolves the name hash through the environment chain (closure env → module env → global env). **An undefined global yields `nil` — no raise.** Traps: —

`OP_SET_GLOBAL` `<key:u16>` — 3 bytes. Stack `v →`. Rebinds the nearest existing binding (may rebind through parent environments). Scalars copied. Assigning over a `const` binding raises `ConstAssign` softly — **the store still happens** and the exception arrives at the next checkpoint (§3.4).

`OP_GET_LOCAL` `<slot:u8>` / **`OP_GET_LOCAL0`** … **`OP_GET_LOCAL3`** — 2 / 1 bytes. Stack `→ v`. Pushes `local[slot]` (slots 0–3 have dedicated 1-byte forms). Traps: —

`OP_SET_LOCAL` `<slot:u8>` / **`OP_SET_LOCAL0`** … **`OP_SET_LOCAL3`** — 2 / 1 bytes. Stack `v →`. Stores into the slot, releasing the previous occupant; scalars copied (value semantics). Writing a slot ≥ the frame's live-local count extends the count. Traps: —

`OP_INC_LOCAL` / **`OP_DEC_LOCAL`** `<slot:u8>` — 2 bytes. Stack `→`. `local[slot] ± 1` in place. Heap-boxed ints and floats mutate their payload without allocating; small ints re-tag. Traps: `InvalidArgs` on non-numeric.

`OP_NIL_LOCAL` `<slot:u8>` — 2 bytes. Stack `→`. Releases `local[slot]` and clears it; a slot already empty is left alone. Emitted at a loop back-edge for locals the body declared, so iteration N does not construct its value while iteration N-1 is still referenced from the slot — `OP_SET_LOCAL` releases the outgoing value only once the incoming one exists, which otherwise keeps two generations of a per-iteration structure live at the same time. Clearing is not observable: reading a body local before its declaration yields nil regardless. Traps: —

7.6 Local compound arithmetic

Shared semantics: `aop` is an `ArithOp` (§6.3) with the exact lane rules of §7.2; all forms are zero-stack except `OP_ARITH_L` / `OP_ARITH_IMM8`. Traps: `DivisionByZero` / `ModuloByZero` / `InvalidArgs` from the arithmetic lanes; `RuntimeError` on an out-of-range `aop` that load-validation missed.

`OP_ARITH_LC` `<dst:u8> <aop:u8> <imm:i8>` — 4 bytes. `local[dst] = local[dst] aop imm`.

`OP_ARITH_LL` `<dst:u8> <aop:u8> <src:u8>` — 4 bytes. `local[dst] = local[dst] aop local[src]`.

`OP_ARITH_L` `<dst:u8> <aop:u8>` — 3 bytes. Stack `v →`. `local[dst] = local[dst] aop v`. For `aop = ADD` with a string destination that is uniquely referenced, the append happens **in place** (this is what keeps `s += x` loops linear); an int right operand appends its decimal text without an intermediate string.

`OP_ARITH_IMM8` `<aop:u8> <imm:i8>` — 3 bytes. Stack `v → r`. `r = v aop imm`.

`OP_ARITH_FMA_LLL` `<dst:u8> <a:u8> <b:u8> <c:u8>` — 5 bytes. Stack `→`. `local[dst] = local[a]·local[b] + local[c]`. All-int lane wraps in int64; otherwise all three operands must be numeric and the result is the double `a·b + c` (computed as separate multiply and add, not a fused C `fma`). Traps: `TypeMismatch` if any operand is non-numeric (note: the old `dst` value is released before the check — after this trap the slot must be treated as dead until reassigned).

7.7 Property and index access (generic)

`OP_GET_PROPERTY` `<key:u16>` — 3 bytes. Stack `obj → v`.

- object/instance/exception receiver: hash lookup; a method value is pushed as a **bound method** (receiver captured);
- class receiver: as above, but reading a non-static method raises `ClassNonStaticCall`;
- module receiver: export lookup; a missing export raises `RuntimeError` ("Import error…");
- any other receiver, or a missing property: pushes `nil`, **no raise**.

`OP_SET_PROPERTY` `<key:u16>` — 3 bytes. Stack `obj, v →`. Hard-checks: non-object-like receiver → `NullPointer`; `const` receiver → `ConstAssign` (no mutation); **instance receivers are sealed** — the key must be a declared field of the class, else `FieldUndeclared`. Scalars copied on store; the replaced value is released.

`OP_GET_INDEX` — 1 byte. Stack `c, k → v`. Dispatch on container type:

- array + int: negative index counts from the end; out-of-bounds raises `IndexOutOfBounds`;
- block + int: bounds-checked, sign-extended read of one element (element size 1/2/4/8 bytes) → int;
- object + string: property value or `nil`;
- string + int: one **byte** as a char (negative index from end; OOB raises);
- `nil[k]`: `nil`, no raise;
- any other container: console error, pushes `nil`, **no raise**.

`OP_SET_INDEX` — 1 byte. Stack `c, k, v →`.

- array + int: a typed array checks the element type (`TypeMismatch`); `k < 0` (after from-end adjust) or `k ≥ 10 000 000` raises `IndexOutOfBounds`; **writing past the end grows the array** to `length = k + 1`;
- block + int: strict bounds (no growth), truncating element store;
- object + string: property store;
- string + int: strict bounds, stores one byte, invalidates the string's cached hash;
- otherwise: `NullPointer`. Const containers: soft `ConstAssign` (§3.4).

7.8 Array fast paths

Fusion hierarchy for reads (fastest first): `GET_ARR_LC` (literal index) → `GET_ARR_LL` (index in local) → `GET_ARR_LI` (int-typed index on TOS) → `GET_INDEX_LOCAL` (any key on TOS) → `GET_INDEX` (fully generic). All are semantically `OP_GET_INDEX` with operands sourced as noted; each falls back to the generic path when the receiver local is not an array, so non-array receivers (string/block/object) still work through them.

OP_GET_ARR_LC `<arr:u8> <idx:u8>` — 3 bytes. Stack `→ v`. `v = local[arr][idx]`, `idx` a 0–255 literal.

OP_GET_ARR_LL `<arr:u8> <idx:u8>` — 3 bytes. Stack `→ v`. `v = local[arr][local[idx]]`.

OP_GET_ARR_LI `<arr:u8>` — 2 bytes. Stack `k → v`. Index popped from TOS (compiler-proven int).

OP_GET_INDEX_LOCAL `<arr:u8>` — 2 bytes. Stack `k → v`. Full generic indexing with the container from a local.

OP_SET_ARR_LC `<arr:u8> <idx:u8>` — 3 bytes. Stack `v →`. `local[arr][idx] = v` (grows like `OP_SET_INDEX`).

OP_SET_ARR_LL `<arr:u8> <idx:u8>` — 3 bytes. Stack `v →`. `local[arr][local[idx]] = v`.

OP_SET_ARR_LLC `<arr:u8> <idx:u8> <src:u8>` — 4 bytes. Stack `→`. `local[arr][idx-literal] = local[src]`, zero stack.

OP_SET_ARR_LLL `<arr:u8> <idx:u8> <src:u8>` — 4 bytes. Stack `→`. `local[arr][local[idx]] = local[src]`, zero stack.

OP_SET_INDEX_LOCAL `<arr:u8>` — 2 bytes. Stack `k, v →`. Generic indexed store, container from a local.

OP_ARITH_ELC `<arr:u8> <idx:u8> <aop:u8> <imm:i8>` — 5 bytes. Stack `→`. `local[arr][local[idx]] aop= imm`. Requires an array receiver and int index; **in-bounds only** (never grows). Traps: `InvalidArgs`, `IndexOutOfBounds`, plus arithmetic-lane traps.

OP_ARITH_ELL `<arr:u8> <idx:u8> <aop:u8> <src:u8>` — 5 bytes. As `OP_ARITH_ELC` with `local[src]` as the right operand.

OP_LOCAL_ARR_LC `<dst:u8> <arr:u8> <idx:u8>` — 4 bytes. Stack `→`. `local[dst] = local[arr][idx-literal]`, zero stack. Traps: `NullPointer` (non-array), `IndexOutOfBounds`.

OP_LOCAL_ARR_LL `<dst:u8> <arr:u8> <idx:u8>` — 4 bytes. Stack `→`. `local[dst] = local[arr][local[idx]]` (array + int index required; negative index from end). Traps: `NullPointer`, `IndexOutOfBounds`.

7.9 Object property fast paths

OP_SET_OBJ_L `<obj:u8> <key:u16>` — 4 bytes. Stack `v →`. `local[obj].key = v`. Same checks as `OP_SET_PROPERTY`, and `const` is **hard-checked** here (no mutation on `ConstAssign`). Traps: `NullPointer`, `ConstAssign`, `FieldUndeclared`.

OP_SET_OBJ_LL `<obj:u8> <key:u16> <src:u8>` — 5 bytes. Stack `→`. `local[obj].key = local[src]`, zero stack; same checks.

7.10 `this` access

`this` is `local[0]` of a method frame. All `this.*` forms raise `NullPointer` when `this` is nil or absent. The `_PROP` / `_ARITH_L/C` / `_INVOKE` forms are the compiler-emitted, hash-keyed instructions; each self-patches to its slot form on first execution (§3.6).

OP_GET_THIS_PROP `<key:u16>` — 3 bytes. Stack `→ v`. On an instance: resolves the member — field → pushes the field (patches to `OP_GET_THIS_SLOT`); class const → pushes the shared value (patches to

`OP_GET_THIS_CONST`); method → pushes a bound method; missing → `nil`. On a plain-object `this`: hash/proto-chain lookup, methods bound, missing → nil.

`OP_SET_THIS_PROP` `<key:u16>` — 3 bytes. Stack `v →`. Instance: key must be a declared field (`FieldUndeclared`), patches to `OP_SET_THIS_SLOT`, stores into the field slot. Plain object: property store. Const `this`: soft `ConstAssign`.

`OP_THIS_ARITH_L` `<key:u16> <aop:u8> <src:u8>` — 5 bytes. Stack `→`. `this.key aop= local[src]`. Patches to `OP_THIS_ARITH_SLOT_L`. Traps: `NullPointer`, `FieldUndeclared`, arithmetic-lane traps.

`OP_THIS_ARITH_C` `<key:u16> <aop:u8> <imm:i8>` — 5 bytes. As above with an i8 immediate; patches to `OP_THIS_ARITH_SLOT_C`.

`OP_GET_THIS_SLOT` `<slot:u16>` — 3 bytes. Stack `→ v`. Pushes instance field `slot` directly. The operand is a resolved field slot, **not** a constant index; guarded at runtime (`this` must be a real instance and `slot < fieldCount`, else `NullPointer`/`RuntimeError`).

`OP_SET_THIS_SLOT` `<slot:u16>` — 3 bytes. Stack `v →`. Stores into the field slot (same guard; soft `ConstAssign` on a const instance).

`OP_GET_THIS_CONST` `<memberIdx:u16>` — 3 bytes. Stack `→ v`. Pushes the shared class-const member. On a guard failure the class layout is rebuilt once before raising (`NullPointer`/`RuntimeError`).

`OP_THIS_ARITH_SLOT_L` `<slot:u16> <aop:u8> <src:u8>` — 5 bytes and **`OP_THIS_ARITH_SLOT_C`** `<slot:u16> <aop:u8> <imm:i8>` — 5 bytes. Compound assign directly on the field slot; same guard, same arithmetic lanes.

`OP_GET_THIS_ARR_L` `<key:u16> <ilocal:u8>` — 4 bytes. Stack `→ v`. `v = this.key[local[ilocal]]` — fused field read + index (peephole fusion of `GET_THIS_PROP + GET_LOCAL + GET_INDEX`). Patches to `OP_GET_THIS_ARR_SLOT_L`. Indexing follows `OP_GET_INDEX` exactly (including nil → nil). Traps: `NullPointer` + indexing traps.

`OP_GET_THIS_ARR_SLOT_L` `<slot:u16> <ilocal:u8>` — 4 bytes. Patched form of the above (field slot resolved).

`OP_BIND_THIS` — 1 byte. Stack `f → bm`. Pushes a bound method binding the current `this` (`local[0]`) to `f`. Used for anonymous functions declared in method bodies. Traps: —

7.11 Construction

`OP_BUILD_OBJECT` `<count:u16>` — 3 bytes. Stack `k₁, v₁, … k_n, v_n → obj`. Pops `count` key/value pairs (value above its key) and builds a fresh object. Scalar values copied. Traps: `StackError` (underflow), `NestingError` (result nesting ≥ 64 deep).

`OP_BUILD_ARRAY` `<count:u16> <elemType:type:u16>` — 5 bytes. Stack `v_n, …, v₁ → arr`. Pops `count` values, TOS-first, into elements `0, 1, …, count−1` — the compiler pushes elements in **reverse source order**, so `[a, b, c]` ends up in source order. `elemType` = 0 → untyped array; `elemType = VAL_FLOAT` → flat float storage (each element coerced with `AsFloat`); any other non-zero `elemType` → typed array — a value whose type doesn't intersect the mask raises `TypeMismatch`. Traps: `StackError`, `TypeMismatch`, `NestingError`.

`OP_MAKE_CONST` — 1 byte. Stack `v → v`. Marks the (heap) value on TOS immutable in place; immediates are already immutable. Traps: —

`OP_NEW` `<argc:u8>` — 2 bytes. Stack `arg₁ … arg_n, class → inst`. Pops the class (else `RuntimeError`), links its inheritance chain, creates a slot-backed instance with every declared field initialized from its default (deep-cloned), then runs the constructor if the class chain has one: a bytecode constructor is invoked as a bound method with the given args (its return value is discarded); a native constructor is called directly. Arity mismatch raises `InvalidArgs`. The instance is pushed. Traps: `RuntimeError`, `InvalidArgs`, plus anything the constructor raises.

`OP_SUPER` — 1 byte. Stack `→ bm`. Pushes `this` bound to the `Constructor` of the **base of the defining class** (`frame->owner`'s proto) — the explicit `super(...)` call then goes through `OP_CALL`. Traps: `NullPointer` (no usable `this`, no owner, no base, or base has no constructor).

`OP_GUARD` — 1 byte. Stack `v → v` (kept) or `v →` + raise. If TOS is nil: pops it and raises `GuardNull`; otherwise leaves the value. Compiled from guard/`!` expressions. Traps: `GuardNull`.

7.12 Calls and returns

Call-frame mechanics, arity and typed-parameter rules: §3.2.

`OP_CALL` `<argc:u8>` — 2 bytes. Stack `arg₁ … arg_n, f → r`. Calls `f`. Plain function → new frame (args become `local[0..argc-1]`, omitted optionals nil-filled). Bound method → receiver becomes `local[0]`, args shift up. Builtin → executed inline, argument types validated. Class → constructor sugar. Async function → spawns a fiber and pushes the **fiber object** immediately (not scheduled until awaited/run). Non-callable → console error, pushes nil, **no raise**. Traps: `StackError` (frame budget), `TypeMismatch` (empty callee slot), `InvalidArgs` (arity/types).

`OP_CALL_TYPED` `<argc:u8>` — 2 bytes. `OP_CALL` minus the runtime argument-type validation (the compiler proved every argument type). Arity is still enforced. Slow-path callees (bound/async/builtin/…) revalidate as in `OP_CALL`.

`OP_CALL_SELF` `<argc:u8>` — 2 bytes. Stack `arg₁ … arg_n → r`. Self-recursive call: the callee is the current frame's function — no lookup, no callee on the stack. Traps: `RuntimeError` (no current function), `StackError`, `InvalidArgs`.

`OP_TAIL_SELF` `<argc:u8>` — 2 bytes. Stack `arg₁ … arg_n →` (frame restarts). Tail self-call: rebinds the current frame's locals to the new arguments and restarts at ip 0. Constant stack and frame depth. Traps: — (argument shapes are the same function's).

`OP_TAIL_CALL` `<argc:u8>` — 2 bytes. Stack `arg₁ … arg_n, f → r`. Fused call+return. A plain function callee **replaces** the current frame (true tail-call elimination — constant frame depth). Any other callee degrades to call-then-return. Traps: as `OP_CALL`.

`OP_RETURN` — 1 byte. Stack (callee) `r →` / (caller) `→ r`. Returns TOS to the caller; tears down the frame (§3.2). Returning from the last frame finishes the fiber (§3.3).

`OP_RETURN_NONE` — 1 byte. Returns **no value**; used as the terminator of top-level/module chunks (function bodies always return a value).

`OP_RETURN_NIL` — 1 byte. Pushes nil, then behaves as `OP_RETURN`. The implicit function epilogue.

`OP_RETURN_L` `<slot:u8>` — 2 bytes. Returns `local[slot]` (fused `GET_LOCAL + RETURN`).

`OP_FN` `<fnIdx:u16>` — 3 bytes. Stack `→ f`. Pushes the function constant after re-pointing its environment at the current environment. No capture, no clone — used for functions that don't close over locals. Traps: `RuntimeError` (constant is not a function).

`OP_FN_CAPTURE` `<fnIdx:u16> <count:u8> count×(<name:u16> <parentSlot:u8>)` — 4 + 3·count bytes. Stack `→ f`. Closure creation: clones the function template, creates a child environment of the current one, and snapshots each captured local (`local[parentSlot]`) into it under the given name. Captures are **by value-reference, at creation time**: rebinding the outer local later is invisible to the closure; mutating a captured container is visible. Inside the body, captured names resolve via `OP_GET_GLOBAL` through the closure environment. Traps: `RuntimeError`.

`OP_YIELD` — 1 byte. Stack `v →`. Suspends the fiber. If a resumer/awaiter is attached, `v` is delivered as its resume value; otherwise `v` is discarded. An awaited generator is re-queued; an un-awaited one parks. Traps: —

`OP_AWAIT` — 1 byte. Stack `v → r`. `v = nil`: plain suspension (resumes with whatever is delivered). `v` a finished fiber: pushes its preserved result immediately, no suspension. `v` a live fiber: parks this fiber until the target completes; the target's result becomes `r` (a fiber that dies without a result delivers nil). Traps: `InvalidArgs` (non-nil non-fiber operand).

7.13 Method invocation

`OP_INVOKE` `<key:u16> <argc:u8>` — 4 bytes. Stack `arg₁ … arg_n, obj → r`. `obj.method(args)`. Resolution: object-likes by hash (plain objects also fall back to the `Object.*` builtin namespace); modules by export; string/char/array receivers sugar into the `String.*` / `Char.*` / `Array.*` builtin with the receiver as first argument. **A missing method drains the args and pushes nil — no raise.** A property that holds a non-function: console error + nil. Builtin methods validate arity and types (`InvalidArgs`). Traps: `RuntimeError` (module without exports), `InvalidArgs`, plus callee raises.

`OP_THIS_INVOKE` `<key:u16> <argc:u8>` — 4 bytes. Stack `arg₁ … arg_n → r`. `this.method(args)`; resolves on the receiver's class chain, honors overrides, self-patches to `OP_THIS_INVOKE_SLOT` when the name resolves to a method of the defining class. Traps: `NullPointer` (unusable `this`), plus `OP_INVOKE` semantics on fallback.

`OP_THIS_INVOKE_SLOT` `<memberIdx:u16> <argc:u8>` — 4 bytes. Patched vtable form: `memberIdx` indexes the receiver class's base-first member layout; the member reference is re-read every call, so subclass overrides and hot-patched functions take effect without re-patching. Guard failure rebuilds the layout once, then raises (`NullPointer` / `RuntimeError`).

`OP_INVOKE_INSTANCE` `<key:u16> <argc:u8> <cache:u16>` — 6 bytes. Stack `arg₁ … arg_n, obj → r`. `OP_INVOKE` specialized for receivers the analyzer proved to be instances (else `RuntimeError`). The trailing u16 is an **inline cache**, rewritten at runtime: a cached layout index is trusted only after its name-hash matches, so call sites shared by different classes stay correct. Semantics otherwise identical to `OP_INVOKE` on an instance.

`OP_INVOKE_GLOBAL` `<globalKey:u16> <methodKey:u16> <argc:u8>` — 6 bytes. Stack `arg₁ … arg_n → r`. Fused `GET_GLOBAL + INVOKE` for `Identifier.method(args)` on a global receiver (imported module, class statics). The receiver never touches the operand stack.

OP_SUPER_INVOKE `<key:u16> <argc:u8>` — 4 bytes. Stack `arg₁ … arg_n → r`. `super.method(args)`: resolves on the **base of the defining class** (never the receiver's dynamic class — this is what makes an overriding method able to call the overridden one without recursing). Traps: `NullPointer` (unusable `this`/owner/base), `RuntimeError` (method not found on the base — no dynamic fallback).

OP_CALL_GLOBAL `<globalKey:u16> <argc:u8>` — 4 bytes. Stack `arg₁ … arg_n → r`. Fused `GET_GLOBAL + CALL`; the callee is resolved from the environment and never pushed. Produced by the peephole pass **after** tail-call optimization, so tail sites keep TCO. An undefined global resolves to nil → "call to non-callable" console error + nil result. Traps: as `OP_CALL`.

7.14 Builtin call shortcuts

OP_PUSH_BUILTIN `<mod:u8> <fn:u8>` — 3 bytes. Stack `→ f`. Pushes builtin function `fn` of builtin module `mod` (indices into the builtin registry, validated at load). Traps: —

OP_CALL0_BUILTIN … OP_CALL5_BUILTIN `<mod:u8> <fn:u8>` — 3 bytes. Stack `arg₁ … arg_N → r` (N = 0…5). Fused builtin call: pops N args, validates arity and declared argument types (`InvalidArgs`; nil passes any type), runs the builtin, pushes its result — a void builtin pushes nothing (the compiler accounts for this statically). Equivalent to `PUSH_BUILTIN + CALL N` without the callee round-trip. Traps: `InvalidArgs`, plus whatever the builtin raises.

7.15 Exception handling

The state machine these instructions drive is specified in §3.4.

OP_TRY_BEGIN `<catchIp:addr:u16> <catchSlot:u8> <finallyIp:addr:u16>` — 6 bytes. Stack `→`. Pushes a try context (state captured: frame count, stack depth, live locals). `catchIp`/`finallyIp` are absolute; `catchSlot` is where a caught exception will bind. Traps: `RuntimeError` (> 24 nested contexts).

OP_TRY_END — 1 byte. Normal try-body completion: jumps to the finally landing pad. Traps: —

OP_CATCH_BEGIN — 1 byte. Stack `e →`. Binds the delivered exception (pushed by the delivery machinery) into `local[catchSlot]`. Traps: `RuntimeError` (no active context — crafted bytecode).

OP_CATCH_END — 1 byte. Normal catch completion: jumps to the finally pad. Traps: `RuntimeError` (no context).

OP_FINALLY_BEGIN — 1 byte. Marks the context in-finally. Traps: `RuntimeError` (no context).

OP_FINALLY_END — 1 byte. Pops the context and resumes its pending action: none → fall through; raise → re-raise the saved exception; return/break/continue → chain outward through enclosing finallys, then return the saved value / jump to the saved target (§3.4). Traps: — (re-raise aside).

OP_TRY_LEAVE `<kind:u8> <count:u8> <target:addr:u16>` — 5 bytes. Stack: kind 0 pops the return value; kinds 1/2 pop nothing. Records a pending `return` (0), `break` (1) or `continue` (2) that must first run `count` levels of enclosing finally bodies, then jumps to the innermost finally. `target` is the break/continue destination. Executed inside a finally body (crafted bytecode; the compiler never emits it there): `RuntimeError`.

OP_THROW — 1 byte. Stack `e →`. Raises `e`, which MUST be an instance whose class chain reaches the builtin `Exception` class, else `TypeMismatch`. Delivery per §3.4; uncaught → process exit.

7.16 Iteration

`OP_FOREACH <valueSlot:u8> <keySlot:u8> <iterSlot:u8> <indexSlot:u8> <exit:addr:u16>` — 7 bytes. Stack `→`. One iteration step; the loop's back-edge jumps back to this instruction. `local[indexSlot]` holds the cursor, initialized to −1 before the loop. Slot operands may be `0xFF` = "loop declares no such variable". Per container in `local[iterSlot]`:

- array: advance index; exhausted when `index ≥ length`; value = element, key = index;
- string: as array over **bytes**; value = char;
- object / instance / exception / class: advance an opaque map cursor (iteration order is the hash map's internal order — NOT insertion order; structural mutation during iteration is undefined); value = entry value, key = entry key; a slot-backed instance with no dynamic map is immediately exhausted;
- other: raise `InvalidArgs`. On exhaustion: value/key slots are set to nil and control jumps to the absolute `exit` address; otherwise the slots are filled and control falls through into the loop body.

`OP_ITER_BEGIN <slot:u8> <endSlot:u8> <exit:addr:u16>` — 5 bytes. Stack `→`. Runs once per counted `iter` loop. Both `local[slot]` (start) and `local[endSlot]` (end) MUST be ints, else `InvalidArgs`; either may be heap-boxed, so the endpoints are not limited to the tagged-int range. If `start > end`, jumps to `exit` (empty range). Otherwise, a span wider than `MAX_ITER_SPAN` (2^{60} steps) raises `InvalidArgs` — the *width* is refused, never the magnitude, so a short range at any magnitude is legal.

`OP_ITER_NEXT <slot:u8> <endSlot:u8> <body:addr:u16>` — 5 bytes. Stack `→`. If `local[slot] < local[endSlot]`: increments the slot and jumps to the absolute `body` address (quantum-checked back-edge); otherwise falls through. Net effect: the loop variable takes every value from start to end **inclusive**. When both bounds are tagged ints the increment is a pure immediate rewrite; if either is boxed, the step goes through a boxed lane that releases the outgoing counter. Traps: —

7.17 Type operations

All are 1 byte, stack `v → r`.

`OP_LEN` — `r:int` = string byte length / array length / block element count / object property count / instance declared-field count / class method+const count; every other type → 0. Traps: —

`OP_TYPE` — `r:int` = the value's `ObjectType` bit (§4.1). Traps: —

`OP_IS_ARRAY` / `OP_IS_OBJECT` — `r:bool`, exact type test (an instance is **not** `VAL_OBJECT`; a typed array is still `VAL_ARRAY`). Traps: —

`OP_HAS_KEY` — Stack `c, k → r:bool`. object: property presence. array: deep-equality membership test. string container with a **char** key: byte-substring test. (As implemented, a *string* key coerces to a NUL needle and the test degenerates to always-true; only char keys are meaningful on strings.) Other combinations → false. Traps: —

`OP_TO_STRING` — canonical text: strings pass through; int decimal; float `%.17g`-equivalent; `true`/`false`; char as quoted UTF-8; `nil` → `"nil"`; containers/functions → `<array[N]>`, `<object>`, `<fn>`, builtin name, `<module>`, `<exception>`; block → its raw bytes as a string. Traps: —

`OP_TO_INT` — int identity; float truncates toward zero; string parsed as decimal (garbage → 0); bool/char → value; nil → 0; other → `InvalidArgs`.

OP_TO_FLOAT — analogous (`string` via decimal float parse). Traps: `InvalidArgs`.

OP_TO_CHAR — char/int/bool → codepoint; string → first byte (empty → 0); nil → 0; other → `InvalidArgs`.

OP_TO_I8 / **OP_TO_I16** / **OP_TO_I32** — coerce to int, then sign-extend the low 8/16/32 bits. **OP_TO_U8** / **OP_TO_U16** / **OP_TO_U32** — coerce to int, then zero-extend the low bits. Traps: as `OP_TO_INT`.

7.18 Modules

OP_EXPORT `<aliasIdx:u16> <nameIdx:u16> <type:u32>` — 9 bytes. Stack `→`. Binds export `alias` to the value of the binding `name` in the module's **own** environment (imported/builtin names from parent environments cannot be re-exported). `type` is the exported symbol's declared `ValMask`, recorded so importers can read export types statically from the `.flx`; the VM ignores it at run time. Traps: `RuntimeError` (missing local binding, non-string constants).

7.19 Debug

Present only in non-stripped chunks; all are semantically transparent.

OP_DBG_LINE `<line:u16>` — 3 bytes. Sets the frame's current source line (stack traces, debugger line stepping). Traps: —

OP_DBG_FUNC_NAME `<idx:const:u16>` — 3 bytes. Sets the frame's function-name string. Traps: —

OP_DBG_FILE_NAME `<idx:const:u16>` — 3 bytes. Sets the chunk's source-file name. Traps: —

OP_DBG_BREAK `<code:u16>` — 3 bytes. Debugger breakpoint: interactive break in the full build, an informational print in the runtime-only build. Traps: —

7.20 Block (raw memory) fast paths

All eight require `local[blk]` to be a block, else `TypeMismatch`. Reads sign-extend the block's element (element size 1/2/4/8 bytes; other sizes → `RuntimeError`) and push an int; writes truncate an int to the element size. Negative indices count from the end. Out-of-bounds raises `IndexOutOfBounds`; every access is additionally validated against the block's registered memory region (guard failure raises).

OP_GET_BLK_LC `<blk:u8> <idx:u8>` — 3 bytes. Push `blk[idx-literal]`. **OP_GET_BLK_LL** `<blk:u8> <idx:u8>` — 3 bytes. Push `blk[local[idx]]`. **OP_SET_BLK_LC** `<blk:u8> <idx:u8>` — 3 bytes. Stack `v →`. `blk[idx-literal] = v`. **OP_SET_BLK_LL** `<blk:u8> <idx:u8>` — 3 bytes. Stack `v →`. `blk[local[idx]] = v`.
OP_SET_BLK_LLC `<blk:u8> <idx:u8> <src:u8>` — 4 bytes. Zero-stack `blk[idx-literal] = local[src]`.
OP_SET_BLK_LLL `<blk:u8> <idx:u8> <src:u8>` — 4 bytes. Zero-stack `blk[local[idx]] = local[src]`.
OP_LOCAL_BLK_LC `<dst:u8> <blk:u8> <idx:u8>` — 4 bytes. Zero-stack `local[dst] = blk[idx-literal]`.
OP_LOCAL_BLK_LL `<dst:u8> <blk:u8> <idx:u8>` — 4 bytes. Zero-stack `local[dst] = blk[local[idx]]`.

7.21 String concatenation

OP_CONCAT — 1 byte. Stack `a, b → r`. Binary string concatenation: both operands are coerced to text (chars UTF-8-encoded, numbers formatted) and joined into a fresh string. Emitted only when the compiler proves both operands are strings; semantically equal to `OP_ADD`'s string lane without the arithmetic dispatch. Traps: —

`OP_CONCAT_N` `<n:u8>` — 2 bytes. Stack `v₁ … v_n → r`. N-ary string concatenation: pops `n` operands (deepest = leftmost), coerces each as `OP_CONCAT` does, and joins them in one pass — one allocation, one hash, each piece copied exactly once (a binary cascade recopies every prefix). `n` MUST be 2…16 (`MAX_CONCAT_FUSE`); the loader rejects other values. Emitted for `a + b + c` chains whose leaves are all statically string/char; longer chains are compiled as nested fusions. Semantically equal to a left-associative cascade of `OP_CONCAT`. Traps: —

---

8. Loader validation

Validation is **fail-closed**: anything not explicitly modeled is rejected. In the reference implementation every load-time rejection terminates the process with exit code 4 (`EXIT_CODE_ERR_FILE`) — malformed bytecode is never surfaced as a catchable runtime exception. Two deliberate exceptions to "reject": a malformed **bundled dependency** is skipped with an error (the outer program still loads), and a malformed **JIT IR section** is dropped (the function runs interpreted).

8.1 File level

1. File size ≥ 192; header readable in full.
2. `magic == "FLS2"`.
3. `MIN_SUPPORTED_BYTECODE_VERSION ≤ compilerVersion ≤ SYS_VERSION`.
4. Signature policy (§5.3): `BAD` / `STRIPPED` / `UNSUPPORTED` always rejected; `UNSIGNED` / `VALID` rejected under `--require-signed`.
5. `mainSectionStart ≥ 192`; non-bundle: `mainSectionStart == 192`.
6. Section ordering `mainSectionStart ≤ stringPoolStart ≤ stringPoolEnd ≤ codeStart ≤ codeEnd ≤ mainSectionEnd ≤ fileSize`.
7. Main-section length ≥ 1 and ≤ `MAX_CODE_SIZE` (10 MiB).
8. Every read is bounds-checked overflow-safely (compare `n > size - pos`, never `pos + n > size`); a short read is fatal.

8.2 Deserialization level

- String pool: entry `len < MAX_STRING_SIZE`; reference `slot < count`.
- Chunk: nesting depth < 64 (§5.6); chunk magic `0xA5`; version window as above; `consts ≤ 16384`; `1 ≤ codelen ≤ 10 MiB`; debug `localCount ≤ 128`; debug-name and IR lengths within their caps.
- Values: depth < 64; string/array/object/class size caps (§5.6); class member `kind ≤ 2`; object keys, class names and member names must be strings; unknown type tag fatal. Function `arity` / `localCount` are clamped (not rejected) and function flags sanitized (§5.6).

8.3 `ValidateChunk`

Every chunk — freshly compiled or deserialized — MUST pass `ValidateChunk` before execution; it recurses into every function constant's sub-chunk (recursion cap 256). It rejects on: NULL/empty code, a constant table announced but absent, and **string-constant hash collisions** — two string constants whose 64-bit hashes are equal but whose bytes differ (the hash is the sole runtime identity of names, §4.6; identical-byte duplicates are legal).

Pass 1 — instruction decode. Every instruction in `[0, codelen)` is decoded. Rejected: unknown opcode; truncated operands; constant index ≥ `constCount`; key operands whose constant is not a real heap string;

local slot ≥ 128 (`OP_FOREACH` slots may also be the sentinel `0xFF`); argc > 16 on every call/invoke form; `OP_CONCAT_N` count outside 2…16 (`MAX_CONCAT_FUSE`); arithmetic sub-op ≥ 11; relative branch target outside `[0, codelen]` ; absolute target ≥ `codelen` ; `OP_TRY_BEGIN` catchSlot ≥ 128; `OP_JUMP_TABLE` with `max < min` ; builtin references out of the builtin registry's range, and `OP_CALL0..5_BUILTIN` targets that are not builtin functions; `OP_EXPORT` indices out of range. The bytes consumed per opcode are cross-checked against the canonical length table (§6.5); any drift is a reject.

Pass 2 — jump edges. Every branch target collected in pass 1 must land on an instruction **boundary** recorded in pass 1 (jumping into the middle of an instruction is rejected).

Pass 3 — abstract stack simulation. The validator symbolically executes the code, tracking operand-stack depth along all paths:

- Any path whose depth would go negative (stack underflow) is rejected — never clamped. This guarantees the dispatch loop can pop operands without per-instruction underflow checks.
- At merge points the **deeper** depth wins (a safe upper bound).
- Code that can fall off the end of the chunk without a terminating instruction (`RETURN*` , `TAIL_*` , `THROW` , `JUMP` , `LOOP` , `JUMP_TABLE` , `TRY_LEAVE`) is rejected.
- Any opcode pass 1 accepts but pass 3 does not model is rejected (fail closed).
- The maximum depth reached is recorded as the chunk's `maxStack` (clamped to 65535); the VM trusts it to reserve stack space once per call instead of checking per push.

Per-instruction depth deltas (net effect; branch seeds in parentheses):

Delta	Instructions
+1	`NIL` `TRUE` `FALSE` `ONE` `NEG_ONE` `DUP` `SUPER` `CONSTANT` `IMM8/16/32` `IMM_F32` `IMM_CHAR` `GET_LOCAL` `GET_LOCAL0-3` `GET_GLOBAL` `GET_THIS_PROP` `GET_THIS_SLOT` `GET_THIS_CONST` `GET_THIS_ARR_L` `GET_THIS_ARR_SLOT_L` `FN` `FN_CAPTURE` `GET_ARR_LC/LL` `GET_LOCAL_PROP` `GET_BLK_LC/LL` `PUSH_BUILTIN` `CALL0_BUILTIN`
0	unary ops, `TO_*` converters, `GET_PROPERTY` `GET_ARR_LI` `GET_INDEX_LOCAL` `ARITH_IMM8/LC/LL/ELC/ELL` `INC/DEC_LOCAL` `NIL_LOCAL` `SET_ARR_LLC/LLL` `SET_OBJ_LL` `THIS_ARITH_*` `LOCAL_ARR_*` `SET_BLK_LLC/LLL` `LOCAL_BLK_*` `DBG_*` `EXPORT` `CALL1_BUILTIN` `ARITH_FMA_LLL` `CATCH_END` `FINALLY_BEGIN/END` `TRY_END` `MAKE_CONST` `GUARD` `BIND_THIS` `YIELD` `AWAIT` `NOP`
−1	`POP` , binary arithmetic and comparisons, `NULL_COALESCING` `HAS_KEY` `GET_INDEX` `CONCAT` `SET_LOCAL` `SET_LOCAL0-3` `SET_GLOBAL` `DEFINE_GLOBAL` `SET_ARR_LC/LL` `SET_OBJ_L` `SET_THIS_PROP` `SET_THIS_SLOT` `SET_BLK_LC/LL` `CALL2_BUILTIN` `ARITH_L` `CATCH_BEGIN`
−2	`SET_PROPERTY` `SET_INDEX_LOCAL` `CALL3_BUILTIN`
−3	`SET_INDEX` `CALL4_BUILTIN`
−4	`CALL5_BUILTIN`
−argc	`CALL` `CALL_TYPED` `NEW` `INVOKE` `INVOKE_INSTANCE` (callee/receiver replaced by result)
1−argc	`INVOKE_GLOBAL` `THIS_INVOKE` `THIS_INVOKE_SLOT` `SUPER_INVOKE` `CALL_GLOBAL` `CALL_SELF` (receiver resolved internally, result pushed)
1−2·count	`BUILD_OBJECT`
1−count	`BUILD_ARRAY`
1−n	`CONCAT_N`

Branch seeding: `JUMP_IF_FALSE/TRUE` continue at `d-1` and seed their target at `d-1` ; `CMP_JUMP_*` continue and seed at `d` ; `TRY_BEGIN` seeds its catch target at `d+1` (the caught exception); `JUMP_TABLE` seeds all targets at `d-1` and terminates the fall-through path; `JUMP` seeds its target at `d` and terminates; `TRY_LEAVE` seeds its target at `d` and terminates; `LOOP` , `TAIL_CALL` , `TAIL_SELF` , `RETURN*` , `THROW` terminate.

Appendix A. Opcode map

Dense numbering for bytecode version 1.0.0.9. `OP_LAST` = 174 (not a real instruction). Any opcode ≥ 174 MUST be rejected.

#	Hex	Mnemonic	#	Hex	Mnemonic
0	0x00	OP_NOP	86	0x56	OP_LOCAL_ARR_LC
1	0x01	OP_CONSTANT	87	0x57	OP_LOCAL_ARR_LL
2	0x02	OP_NIL	88	0x58	OP_BUILD_OBJECT
3	0x03	OP_TRUE	89	0x59	OP_BUILD_ARRAY
4	0x04	OP_FALSE	90	0x5A	OP_MAKE_CONST
5	0x05	OP_ONE	91	0x5B	OP_CALL
6	0x06	OP_NEG_ONE	92	0x5C	OP_CALL_TYPED
7	0x07	OP_IMM8	93	0x5D	OP_CALL_SELF
8	0x08	OP_IMM16	94	0x5E	OP_TAIL_SELF
9	0x09	OP_IMM32	95	0x5F	OP_TAIL_CALL
10	0x0A	OP_IMM_F32	96	0x60	OP_RETURN
11	0x0B	OP_IMM_CHAR	97	0x61	OP_RETURN_NONE
12	0x0C	OP_POP	98	0x62	OP_RETURN_NIL
13	0x0D	OP_DUP	99	0x63	OP_RETURN_L
14	0x0E	OP_NEGATE	100	0x64	OP_FN
15	0x0F	OP_NOT	101	0x65	OP_FN_CAPTURE
16	0x10	OP_BITWISE_NOT	102	0x66	OP_YIELD
17	0x11	OP_ADD	103	0x67	OP_AWAIT
18	0x12	OP_SUBTRACT	104	0x68	OP_INVOKE
19	0x13	OP_MULTIPLY	105	0x69	OP_THIS_INVOKE
20	0x14	OP_DIVIDE	106	0x6A	OP_BIND_THIS
21	0x15	OP_MODULO	107	0x6B	OP_PUSH_BUILTIN
22	0x16	OP_POWER	108	0x6C	OP_CALL0_BUILTIN
23	0x17	OP_BITWISE_AND	109	0x6D	OP_CALL1_BUILTIN
24	0x18	OP_BITWISE_OR	110	0x6E	OP_CALL2_BUILTIN
25	0x19	OP_XOR	111	0x6F	OP_CALL3_BUILTIN
26	0x1A	OP_SHL	112	0x70	OP_CALL4_BUILTIN
27	0x1B	OP_SHR	113	0x71	OP_CALL5_BUILTIN
28	0x1C	OP_IS_NIL	114	0x72	OP_TRY_BEGIN
29	0x1D	OP_IS_NOT_NIL	115	0x73	OP_TRY_END
30	0x1E	OP_EQUAL	116	0x74	OP_CATCH_BEGIN
31	0x1F	OP_NOT_EQUAL	117	0x75	OP_CATCH_END
32	0x20	OP_LESS	118	0x76	OP_FINALLY_BEGIN
33	0x21	OP_LESS_EQUAL	119	0x77	OP_FINALLY_END
34	0x22	OP_GREATER	120	0x78	OP_TRY_LEAVE
35	0x23	OP_GREATER_EQUAL	121	0x79	OP_THROW
36	0x24	OP_APPROX_EQ	122	0x7A	OP_FOREACH
37	0x25	OP_CMP_IS	123	0x7B	OP_ITER_BEGIN
38	0x26	OP_JUMP_IF_FALSE	124	0x7C	OP_ITER_NEXT
39	0x27	OP_JUMP_IF_TRUE	125	0x7D	OP_LEN
40	0x28	OP_JUMP	126	0x7E	OP_TYPE
41	0x29	OP_LOOP	127	0x7F	OP_IS_ARRAY
42	0x2A	OP_NULL_COALESCING	128	0x80	OP_IS_OBJECT
43	0x2B	OP_JUMP_TABLE	129	0x81	OP_HAS_KEY
44	0x2C	OP_CMP_JUMP_LL	130	0x82	OP_TO_STRING
45	0x2D	OP_CMP_JUMP_LC	131	0x83	OP_TO_INT

#	Hex	Mnemonic	#	Hex	Mnemonic
46	0x2E	OP_DEFINE_GLOBAL	132	0x84	OP_TO_FLOAT
47	0x2F	OP_GET_GLOBAL	133	0x85	OP_TO_CHAR
48	0x30	OP_SET_GLOBAL	134	0x86	OP_TO_I8
49	0x31	OP_GET_LOCAL	135	0x87	OP_TO_U8
50	0x32	OP_SET_LOCAL	136	0x88	OP_TO_I16
51	0x33	OP_GET_LOCAL0	137	0x89	OP_TO_U16
52	0x34	OP_GET_LOCAL1	138	0x8A	OP_TO_I32
53	0x35	OP_GET_LOCAL2	139	0x8B	OP_TO_U32
54	0x36	OP_GET_LOCAL3	140	0x8C	OP_EXPORT
55	0x37	OP_SET_LOCAL0	141	0x8D	OP_NEW
56	0x38	OP_SET_LOCAL1	142	0x8E	OP_SUPER
57	0x39	OP_SET_LOCAL2	143	0x8F	OP_GUARD
58	0x3A	OP_SET_LOCAL3	144	0x90	OP_DBG_LINE
59	0x3B	OP_INC_LOCAL	145	0x91	OP_DBG_FUNC_NAME
60	0x3C	OP_DEC_LOCAL	146	0x92	OP_DBG_FILE_NAME
61	0x3D	OP_ARITH_LC	147	0x93	OP_DBG_BREAK
62	0x3E	OP_ARITH_LL	148	0x94	OP_GET_BLK_LC
63	0x3F	OP_ARITH_L	149	0x95	OP_GET_BLK_LL
64	0x40	OP_ARITH_IMM8	150	0x96	OP_SET_BLK_LC
65	0x41	OP_GET_PROPERTY	151	0x97	OP_SET_BLK_LL
66	0x42	OP_SET_PROPERTY	152	0x98	OP_SET_BLK_LLC
67	0x43	OP_GET_INDEX	153	0x99	OP_SET_BLK_LLL
68	0x44	OP_SET_INDEX	154	0x9A	OP_LOCAL_BLK_LC
69	0x45	OP_GET_ARR_LC	155	0x9B	OP_LOCAL_BLK_LL
70	0x46	OP_GET_ARR_LL	156	0x9C	OP_CONCAT
71	0x47	OP_GET_ARR_LI	157	0x9D	OP_ARITH_FMA_LLL
72	0x48	OP_GET_INDEX_LOCAL	158	0x9E	OP_INVOKE_INSTANCE
73	0x49	OP_SET_ARR_LC	159	0x9F	OP_CMP_JUMP_LC32
74	0x4A	OP_SET_ARR_LL	160	0xA0	OP_INVOKE_GLOBAL
75	0x4B	OP_SET_ARR_LLC	161	0xA1	OP_GET_LOCAL_PROP
76	0x4C	OP_SET_ARR_LLL	162	0xA2	OP_GET_THIS_SLOT
77	0x4D	OP_SET_INDEX_LOCAL	163	0xA3	OP_SET_THIS_SLOT
78	0x4E	OP_ARITH_ELC	164	0xA4	OP_GET_THIS_CONST
79	0x4F	OP_ARITH_ELL	165	0xA5	OP_THIS_ARITH_SLOT_L
80	0x50	OP_SET_OBJ_L	166	0xA6	OP_THIS_ARITH_SLOT_C
81	0x51	OP_SET_OBJ_LL	167	0xA7	OP_THIS_INVOKE_SLOT
82	0x52	OP_GET_THIS_PROP	168	0xA8	OP_GET_THIS_ARR_L
83	0x53	OP_SET_THIS_PROP	169	0xA9	OP_GET_THIS_ARR_SLOT_L
84	0x54	OP_THIS_ARITH_L	170	0xAA	OP_SUPER_INVOKE
85	0x55	OP_THIS_ARITH_C	171	0xAB	OP_CALL_GLOBAL
—	—	—	172	0xAC	OP_CONCAT_N
—	—	—	173	0xAD	OP_NIL_LOCAL

Appendix B. Machine limits and named constants

This appendix is the normative value table for **every** named constant used in this document — the specification is self-contained and requires no access to the VM's source code. A conforming implementation MUST enforce the same caps where they are load-time validation rules, and SHOULD match them at run time for portability of programs near the limits.

B.1 Format constants

Constant	Value	Meaning
`FLS_HEADER_SIZE`	192	`.flx` file-header size in bytes (§5.2)
`CHUNK_MAGIC`	`0xA5`	first byte of every serialized chunk (§5.5)
`FLX_POOL_REF`	`0xFFFFFF01`	type-tag word marking a string-pool reference (§5.4)
`FLX_POOL_MAX`	65535	maximum string-pool entries per code region (§5.4)
`BUNDLE_ENTRY_SIZE`	152	bundle TOC entry size in bytes (§5.7)
`MAX_BUNDLE_DEPS`	1024	maximum dependencies in one bundle TOC (§5.7)
`FLS_SIG_CONTEXT`	`"flaris.flx.sig.v1"`	signature domain-separation string (§5.3)
`XXHASH64_SEED`	0	seed for all identifier hashing (§4.6)
`XXHASH64_EMPTY`	`0xEF46DB3751D8E999`	xxHash64 of the empty string (§4.6)
`EXIT_CODE_ERR_FILE`	4	process exit code for every load-time rejection (§8)
`MAX_VALIDATE_NESTING`	256	`ValidateChunk` recursion cap over nested function chunks (§8.3)

B.2 Machine limits

Constant	Value	Applies to
`MAX_FIBERS`	1024	live fibers (`MIN_FIBERS` 2, default 256)
`MAX_STACK`	65535	operand-stack slots per fiber (`MIN_STACK` 64, default 4096)
`MAX_CALL_FRAMES`	1024	call depth per fiber (`MIN_CALL_FRAMES` 8, default 64)
`MAX_LOCALS`	128	local slots per frame
`MAX_CALL_ARGUMENTS`	16	arguments per call (+2 internal slots for bound methods)
`MAX_BUILTIN_ARGUMENTS`	6	declared builtin parameters
`MAX_FL_EXCEPTION_LEVELS`	24	nested try contexts per fiber
`MAX_PROTO_NEST`	8	class-inheritance depth
`MAX_CHUNK_DEPTH`	64	nested-chunk depth in a `.flx`
`MAX_CODE_SIZE`	10 MiB	bytecode bytes per chunk
`MAX_CONST_COUNT`	16384	constants per chunk
`MAX_ITEMS`	10 000 000	array elements / object properties
`MAX_STRING_SIZE`	256 MiB	string bytes
`MAX_MEM_BLOCK_ALLOC`	2 GiB	one memory block
`MAX_TOTAL_BLOCK_ALLOCATION`	64 GiB	total block memory
`MAX_BLOCK_ELEM_SIZE`	255	block element size (stored in one byte)
`MAX_EVAL_COMPILE_SRC_LEN`	10 KiB	`Eval` / `Compile` source length
`MAX_CONCAT_FUSE`	16	operands of one `OP_CONCAT_N` (§7.21)
`MAX_NEST_DEPTH`	64	container nesting before `NestingError` (build/merge)
`MAX_COMPARE_DEPTH`	1024	structural-comparison recursion before `NestingError`
`MAX_PRINT_DEPTH`	64	console value printing descends this deep, then elides
`FIBER_QUANTUM`	10000	scheduling checkpoints per time slice
`MAILBOX_CAP`	64	per-fiber mailbox depth
`MAX_EVENTS`	64	event objects
`SSO_MAX_CHARS`	7	small-string optimization threshold (implementation note, §4.4 — not observable)

Appendix C. Worked example

A minimal program (`spec_check.fls`):

```
fn Add(a, b)  { return a + b; }
fn Main()     { let total = 0;
```

```
                for (let i = 0; i < 10; i++) { total += Add(i, 2); }
                Console.WriteLine(total); }
```

compiled with `flarisvm --compile spec_check.fls spec_check.flx` (unsigned) produces a 627-byte file. Annotated prefix (all values little-endian):

```
offs  bytes                     field
0000  46 4C 53 32               magic "FLS2"
0004  00 00 00 01               library version 0x01000000  (1.0.0.0)
0008  08 00 00 01               bytecode version 0x01000008 (1.0.0.8)
000C  73 70 65 63 5F 63 68 65   name "spec_check\0" (basename, no extension)
      63 6B 00 …
002C  00 00 00 00               flags = 0 (not a bundle)
0030  DC BE 50 90 22 3A 78 C6   entry = 0xC6783A229050BEDC = xxhash64("Main")
0038  C0 00 00 00  C0 00 00 00  mainSectionStart = stringPoolStart = 192
0040  FE 00 00 00  FE 00 00 00  stringPoolEnd = codeStart = 254
0048  73 02 00 00  73 02 00 00  codeEnd = mainSectionEnd = 627 (= file size)
0052  00                        sigAlg = 0 (unsigned)
0054  00 × 32                   pubkey (zero)
0074  00 × 64                   signature (zero)
00B4  00 × 12                   padding
                                ── string pool (§5.4) ──
00C0  03 00                     pool count = 3
00C2  00                        [0] isStatic
00C3  57 77 68 5F D7 66 D3 1A   [0] hash = 0x1AD366D75F687757 = xxhash64("Add")
00CB  03 00 00 00  41 64 64     [0] len 3, "Add"
00D2  00                        [1] isStatic
00D3  62 3C 10 EF B6 40 DB 65   [1] hash
00DB  0E 00 00 00  73 70 65 …   [1] len 14, "spec_check.fls"
00EA  00                        [2] isStatic
00EB  DC BE 50 90 22 3A 78 C6   [2] hash = xxhash64("Main")
00F3  04 00 00 00  4D 61 69 6E  [2] len 4, "Main"
                                ── top-level chunk (§5.5) ──
00FE  A5                        chunk magic
00FF  08 00 00 01               chunk version 0x01000008
0103  04 00                     consts = 4
0105  13 00 00 00               codelen = 19
0109  …                         flags, then constants / code / debug per §5.5
```

The three pooled strings are exactly the strings that occur more than once in the value tree ("Add" and "Main" appear as both function name and global key; the source path appears in both functions' debug info) — single-use strings stay inline (§5.4).

Appendix D. Container format diagram

The `.flx` bytecode file format

Magic **FLS2** · little-endian throughout · Ed25519-signable · shared string pool · portable fixed-width integers

1 · File layout (byte offset increases →)

Ed25519 signature covers SHA-256 of the whole file (signature field zeroed)

Header	Bundle TOC	String pool	Code region
192 B, fixed	only if FLAG_BUNDLE	deduped strings	main chunk, with nested function chunks inline

@0 @192

main section = string pool + code · the unit a bundle copies per dependency (offsets @56–@76)

2 · Header — 192 bytes, fixed layout

off	sz	field	
@0	4	magic	"FLS2"
@4	4	version	format M.m.p.b (u32)
@8	4	compiler_version	u32
@12	32	name	output basename, NUL-term ‡
@44	4	flags	bit0 = FLAG_BUNDLE
@48	8	entry	entry-fn hash · 0 = library
@56	4	main_section_start	
@60	4	string_pool_start	
@64	4	string_pool_end	
@68	4	code_start	↑ absolute, validated on load

off	sz	field	
@72	4	code_end	
@76	4	main_section_end	
@80	2	(reserved)	
@82	1	sig_alg	0 = unsigned · 2 = Ed25519
@84	32	pubkey	Ed25519 public key
@116	64	signature	Ed25519 signature
@180	12	(padding → 192)	

Ed25519 signing block (@82–@179)

Signs the domain-separated message
`"flaris.flx.sig.v1" ‖ 0x00 ‖ SHA-256(file, sig=0)`
Trust anchors compiled in + ~/.flaris/trusted_keys

3 · Chunk — recursive (the code region is one chunk; each function constant nests another)

ChunkHeader	constants[consts]	code[codelen]	debug info	JIT IR
13 B	values (see 4)	bytecode · LE operands	if CHUNK_FLAG_DBG	if CHUNK_FLAG_HAS_JIT_IR

`ChunkHeader = magic u8 (0xA5) · version u32 · consts u16 · codelen u32 · flags u16`

flags: DBG = 0x0001 · HAS_JIT_IR = 0x0002. A `function` constant carries a whole nested Chunk — the loader caps nesting at depth 64 (MAX_CHUNK_DEPTH).
A single-byte `is_const` flag follows every value's type tag (see 4); the string pool (5) is read once, before the top chunk.

4 · Constant / value encoding

inline value
`[ type : u32 ] [ is_const : u8 ] [ payload ]`

pool reference → shared string pool (see 5)
`[ 0xFFFFFF01 : u32 ] [ slot : u16 ]`

payload by type

nil	— (no payload)	string	hash u64 · len u32 · bytes[len]
bool	u8	array	count u32 · flat u8 · elements…
char	u32	object	count u32 · (key,value) × count
int	i64	function	arity u8 · locals u8 · flags u32 · ret u32 · arg_types u32×16 · name (value) · nested Chunk
float	f64		

Strings appearing more than once in a file are stored once in the pool and emitted as pool references, shrinking shipped `.flx` ~15–30%.

5 · String pool — every repeated string stored once

Lives at `string_pool_start..string_pool_end` (header @60/@64), inside the main section, read once before the top chunk.

count : u16	entry 0	entry 1	entry … N-1
N entries	slot 0	slot 1	

pool entry (variable length)

`[ is_static : u8 ] [ hash : u64 (xxHash64) ] [ len : u32 ] [ bytes : len ]`

A constant references entry `i` via the pool-reference value `0xFFFFFF01, slot=i` (see 4). `is_static` is advisory (ignored on load); each reference takes its own ref on the shared Object.

6 · Bundle (FLAG_BUNDLE) — a self-contained, multi-module `.flx`

Built with `flarisvm --compile app.fls out.flx --bundle=Logger.flx;Router.flx`. FLAG_BUNDLE (header @44 bit0) is set; the VM pre-loads every embedded dependency before execution — no filesystem access for them. Cap: `MAX_BUNDLE_DEPS = 1024`.

File structure of a bundle (example: app + 2 deps)

Header	TOC	app · main section	Logger · main section	Router · main section	
FLAG_BUNDLE	count=2 + 2 entries	pool + code	dep 0	dep 1	← each entry's offset/size locate its dep section
@0	@192	main_section_start	entry[0].offset	entry[1].offset	

Bundle TOC — at @192, immediately after the header

```
[ entry_count : u16 ] then BundleEntry × entry_count
```

`BundleEntry` — 152 bytes, no padding

`name[64]`	module name, NUL-term, no ".flx"	`sha256[32]`	dep's whole-file SHA-256 — the `library()` pin
`version : u32`	0xMMmmppbb	`sig_alg : u8`	dep's signature scheme (0 = unsigned)
`offset : u64`	absolute offset of dep's main section	`pubkey[32]`	dep's Ed25519 key (zero if unsigned)
`size : u32`	byte length of that section	`reserved[7]`	zero

A bundled dep keeps only its main section (its own header is dropped), so the pin travels in `sha256` here — a bundled and an on-disk dep pin identically.
The region ranges (`offset`, `size`) are bounds-checked against the file before any allocation.

7 · Integrity, byte order & load-time verification

- **Byte order:** little-endian throughout — header, string pool, bytecode operands and JIT IR (native order on x86/ARM, no byte-swap to decode).
- **Pin / fingerprint:** whole-file SHA-256 equals `sha256sum file.flx`; `library(name, version, hash)` enforces it.
- **Signature:** Ed25519 over the domain-separated digest above; trust anchors are compiled in, plus the user's `~/.flaris/trusted_keys`.
- **Verification:** the VM checks operand bounds, jump-target alignment and per-frame stack depth before executing; a bad instruction raises `IllegalInstruction` (16).
- **Portability:** only fixed-width integer types on disk; no pointer-sized fields. Opcode numbering is versioned — an older chunk is rejected with a recompile hint (version floor).

‡ the 32-byte `name` (output basename) is part of the signed / fingerprinted bytes, so renaming the output changes the pin.